



LAUREA MAGISTRALE
UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di **Informatica, Sistemistica e Comunicazione**

Laurea magistrale in **informatica**

Titolo da decidere

Francesco Romeo

Matricola: 885880

Relatrice: **Dott.ssa Elisabetta Fersini**

Co-relatore: **Dott. Marco Loregian**

Anno Accademico **2025/2026**

Sommario

“abstract”

Indice

Introduzione	iii
1 Stato dell'Arte e Background Tecnologico	1
1.1 Architetture Transformer	1
1.1.1 Dalle reti neurali ricorrenti ai Transformer	1
1.1.2 Il meccanismo di Self-Attention	2
1.1.3 Architettura encoder-decoder e varianti decoder-only	2
1.1.4 Limiti in contesto zero-shot e motivazione del RAG	2
1.2 LLM: panorama e analisi comparativa	3
1.2.1 Modelli proprietari	3
1.2.2 Motivazioni per l'adozione di modelli locali	4
1.2.3 Modelli open-weight per uso on-premise	5
1.2.4 Quantizzazione e inferenza on-premise	5
1.2.5 Estensione del contesto: Infini-attention	6
1.2.6 Ollama: server di inferenza locale	6
1.3 Retrieval-Augmented Generation (RAG)	6
1.3.1 Motivazione e definizione	6
1.3.2 Architettura del sistema RAG	6
1.3.3 Modelli di embedding	8
1.3.4 Database vettoriali	9
1.3.5 Agentic RAG	9
1.3.6 Tecniche di adattamento al dominio	9
1.4 Framework di orchestrazione: Haystack	10
1.5 Sintesi e posizionamento del lavoro	11
2 Analisi dei Requisiti e Progettazione del Sistema	12
2.1 Il contesto aziendale: Intré S.r.l.	12
2.1.1 Il modello delle Gilde	13
2.2 Caso d'uso: Proposta di nuove Gilde Intré	13
2.2.1 Motivazione del caso d'uso	14
2.2.2 Obiettivi del caso d'uso	14
2.2.3 Descrizione dei dati e analisi del corpus	14
2.3 Requisiti di sistema	15
2.3.1 Vincoli architetturali	16
2.4 Scelte tecnologiche: analisi comparativa	16

2.4.1	Framework di orchestrazione RAG	17
2.4.2	Database vettoriale	18
2.4.3	Modello di embedding	19
2.4.4	Server di inferenza LLM e selezione del modello di chat	19
2.5	Proposta architetturale	21
2.5.1	Componenti principali	21
2.5.2	Progettazione della collection Qdrant	22
2.5.3	Pipeline di ingestione ETL	23
2.5.4	Pipeline di retrieval e logica di ranking	23
2.5.5	Endpoint API di riferimento	24
2.6	Integrazione con i3Guild	25
2.6.1	Servizi Docker aggiuntivi	25
2.6.2	Variabili d'ambiente	25
2.6.3	Integrazione lato Next.js	25
2.6.4	Sincronizzazione dei dati	26
3	Implementazione del Sistema RAG	27
3.1	Struttura dei repository e responsabilità	28
3.2	Pipeline di ingestione	28
3.2.1	Configurazione e controlli preliminari	28
3.2.2	Estrazione dei dati da PostgreSQL	29
3.2.3	Normalizzazione e costruzione dei documenti	29
3.2.4	Sorgenti aggiuntive	30
3.2.5	Embedding e scrittura in Qdrant	30
3.3	Microservizio RAG	30
3.3.1	Avvio, middleware e gestione errori	31
3.3.2	Endpoint di retrieval	31
3.3.3	Pipeline Haystack di query	31
3.4	Logica di retrieval	32
3.4.1	Riscrittura della query e filtri	32
3.4.2	Cache e deduplicazione	32
3.4.3	Selezione dinamica dei documenti	32
3.5	Generazione con Ollama	33
3.5.1	Risoluzione del modello	33
3.5.2	Costruzione del prompt aumentato	33
3.5.3	Streaming NDJSON	33
3.6	Workflow agentico	34
3.6.1	Stato conversazionale	34
3.6.2	Regole di orchestrazione	35
3.6.3	Eventi di streaming agentico	35
3.7	Integrazione lato Next.js	35
3.7.1	Client di retrieval	35
3.7.2	Client chat e modalità agentica sperimentale	35
3.8	Generazione della bozza di Gilda	36
3.9	Aspetti operativi	36
3.9.1	Timeout e dipendenze lente	36

3.9.2	System prompt e modello derivato	36
3.9.3	Tracciabilità	37
3.10	Sintesi del capitolo	37
4	Ottimizzazione e Adattamento al Dominio	38
5	Analisi Sperimentale e Valutazione dei Risultati	39
	Conclusioni e Sviluppi Futuri	40
A	Annotazioni e Convenzioni	41
A.1	Repository e componenti	41
A.2	Formato dei documenti indicizzati	41
A.3	Eventi NDJSON	41
B	Codice Utilizzato	43
B.1	Listati richiamati nel Capitolo 2	43
B.2	Listati richiamati nel Capitolo 3: pipeline di ingestione	44
B.3	Listati richiamati nel Capitolo 3: retrieval e generazione	46
B.4	Listati richiamati nel Capitolo 3: workflow agentico e frontend	47
	Bibliografia	49
	Ringraziamenti	52

Introduzione

- **Contesto:** Diffusione dei Large Language Models (LLM) e le sfide dell'adozione in ambito corporate (privacy, allucinazioni, aggiornamento dei dati). → giustificare le affermazioni con citazioni e dati.
- **Il Problema:** Gestione della conoscenza non strutturata e valorizzazione del materiale formativo (Gilde) in Intré. → descrizione dell'azienda e dello use-case.
- **Obiettivo della tesi:** Progettazione e sviluppo di un sistema RAG (Retrieval-Augmented Generation) con focus sull'ottimizzazione dell'inferenza e dell'accuratezza nel dominio specifico.
- **Estrema sintesi dei risultati ottenuti**
- **NOTA:** essere esplicativi sotto tutti i punti di vista, deve essere una versione sintetica della tesi - un riassunto - con l'aggiunta delle motivazioni del progetto.

Todo list

Capitolo 1

Stato dell'Arte e Background Tecnologico

Il sistema sviluppato in questa tesi nasce dall'esigenza di rendere interrogabile una base di conoscenza aziendale senza trasferire dati verso servizi esterni. Prima di descriverne i requisiti e l'implementazione, è quindi necessario chiarire il quadro tecnologico entro cui si colloca: i modelli linguistici generativi, le tecniche che ne rendono possibile l'esecuzione locale e il paradigma *Retrieval-Augmented Generation* (RAG), adottato per collegare la generazione testuale a fonti documentali verificabili. Il capitolo procede seguendo questa linea argomentativa: dalla struttura dei Transformer si passa al confronto tra modelli proprietari e open-weight, per poi introdurre RAG, database vettoriali, modelli di embedding e tecniche di adattamento al dominio.

1.1 Architetture Transformer

1.1.1 Dalle reti neurali ricorrenti ai Transformer

Il modello Transformer, introdotto da Vaswani et al. nel 2017 [32], costituisce il punto di svolta da cui derivano gli attuali modelli linguistici. Prima della sua introduzione, gran parte dei sistemi di trattamento del linguaggio naturale (*Natural Language Processing*, NLP) si basava su reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) e, in particolare, su architetture LSTM (*Long Short-Term Memory*) [13]. Tali modelli elaboravano la sequenza un elemento alla volta, aggiornando progressivamente uno stato nascosto. Questa impostazione era coerente con la natura ordinata del testo, ma introduceva due limiti rilevanti: la scarsa parallelizzabilità dell'addestramento e la difficoltà nel preservare dipendenze informative tra elementi molto distanti nella sequenza (*vanishing gradient*).

Il Transformer affronta tali limiti sostituendo la ricorrenza con un meccanismo di *self-attention* globale. In questo modo ogni token può essere messo in relazione diretta con tutti gli altri token della sequenza, indipendentemente dalla loro distanza posizionale. Questa scelta architetturale spiega sia la maggiore efficienza in addestramento sia la capacità dei modelli successivi di costruire rappresentazioni contestuali più ricche.

1.1.2 Il meccanismo di Self-Attention

Il nucleo computazionale del Transformer è il meccanismo di *attention* scalato. Data una sequenza di token rappresentati come vettori di dimensione d_{model} , si calcolano tre proiezioni lineari per ogni elemento: la query $\mathbf{Q}$, la chiave $\mathbf{K}$ e il valore $\mathbf{V}$. Il punteggio di attenzione [32] è definito come:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (1.1)$$

dove d_k è la dimensione dei vettori di chiave; il fattore $1/\sqrt{d_k}$ previene la saturazione della funzione softmax in presenza di prodotti scalari molto grandi.

Il medesimo calcolo viene poi replicato su più *teste* di attenzione (*attention heads*). Ciascuna testa apprende una proiezione diversa dello spazio di input e può quindi specializzarsi su relazioni differenti, ad esempio sintattiche, semantiche o posizionali [32]. La concatenazione degli output di h teste definisce il meccanismo di *Multi-Head Attention*:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (1.2)$$

con $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

La posizione dei token nella sequenza non è catturata dall'attention; viene iniettata tramite *positional encoding*, ossia vettori sommati agli embedding di input che forniscono informazioni sull'ordine relativo degli elementi.

1.1.3 Architettura encoder-decoder e varianti decoder-only

L'architettura originale del Transformer è di tipo encoder-decoder. L'encoder trasforma la sequenza di input in una rappresentazione contestuale densa; il decoder utilizza tale rappresentazione, tramite *cross-attention*, per produrre la sequenza di output. Questa struttura è particolarmente adatta a compiti *sequence-to-sequence*, come la traduzione automatica [32].

I modelli linguistici generativi contemporanei adottano prevalentemente un'architettura *decoder-only*, nella quale l'attenzione è applicata in modo *causale* (o *masked*): ogni token dipende esclusivamente dai precedenti. Questo vincolo è necessario per l'addestramento autoregressivo, nel quale il modello impara a predire il token successivo dato il contesto.

1.1.4 Limiti in contesto zero-shot e motivazione del RAG

La capacità dei Transformer di modellare il linguaggio non elimina tuttavia un problema centrale per l'uso aziendale: un LLM pre-addestrato non conosce, per definizione, i dati interni prodotti dopo l'addestramento e non dispone di un meccanismo nativo per verificarli. In un dominio come quello considerato in questa tesi, dove le risposte devono riferirsi a documenti aziendali specifici, questa caratteristica diventa un limite operativo. La Tabella 1.1 riassume le criticità principali e le relative strategie di mitigazione.

Da questa analisi deriva la scelta di privilegiare il paradigma RAG, discusso nella Sezione 1.3. Rispetto al fine-tuning, RAG permette di aggiornare la conoscenza disponibile intervenendo sul corpus documentale e non sui pesi del modello; per questo risulta più adatto a basi di conoscenza aziendali soggette a modifiche frequenti.

Tabella 1.1: Limiti strutturali degli LLM in contesti aziendali e relative strategie di mitigazione.

Limitazione	Descrizione	Strategia di mitigazione
Conoscenza statica	I parametri codificano la conoscenza fino al <i>knowledge cutoff</i> ; nessun accesso a dati interni o aggiornati dopo l’addestramento.	RAG: recupero dinamico da knowledge base esterna, aggiornabile senza riaddestrare il modello.
Allucinazioni [17]	Il modello può generare testo plausibile ma fattualmente errato su informazioni assenti dalla distribuzione di addestramento.	RAG con <i>grounding</i> : la risposta è ancorata a documenti recuperati e verificabili.
Privacy dei dati	Il fine-tuning su modelli cloud può esporre dati riservati a infrastrutture di terze parti, con rischi di conformità normativa.	Esecuzione on-premise con modelli open-weight: nessuna trasmissione di dati verso servizi esterni.

1.2 LLM: panorama e analisi comparativa

Una volta chiariti i limiti dei modelli generativi, occorre distinguere tra le famiglie di LLM disponibili. A partire dal 2022 l’ecosistema si è diviso lungo una linea rilevante per questa tesi: da un lato i modelli proprietari, accessibili prevalentemente tramite API cloud; dall’altro i modelli open-weight, i cui pesi possono essere scaricati ed eseguiti su infrastruttura locale, nel rispetto delle relative licenze. La Tabella 1.2 offre una panoramica dei modelli considerati come riferimento tecnologico.

Tabella 1.2: Panorama comparativo dei principali LLM (2024–2025). [†] Esecuzione garantita on-premise senza dipendenze cloud; [‡] pesi scaricabili pubblicamente; MoE = *Mixture of Experts*.

Modello	Sviluppatore	Licenza	Accesso	Architettura / Param. attivi	Ctx	Multi-modale
GPT-5 / o3	OpenAI	Proprietaria	API cloud	Dense, n.d.	128k	✓
Claude 4 Sonnet	Anthropic	Proprietaria	API cloud	Dense, n.d.	200k	✓
Gemini 3 Pro	Google DM	Proprietaria	API cloud	Dense, n.d.	1M	✓
Llama 4 Scout	Meta AI	Open-weight [‡]	Self-hosted [†]	MoE, 17B	10M	✓
Mistral Large 3	Mistral AI	Apache 2.0 [‡]	Self-hosted [†]	MoE, ~40B	128k	✓
Qwen3-8B	Alibaba Cloud	Apache 2.0 [‡]	Self-hosted [†]	Dense, 8B	128k	–

1.2.1 Modelli proprietari

I modelli proprietari offrono prestazioni elevate sui benchmark generali, ma presentano vincoli rilevanti per l’adozione in contesti con requisiti di riservatezza.

GPT-5 e OpenAI o-series. La famiglia GPT-5 [27] rappresenta un riferimento per la pianificazione complessa e l’integrazione multimodale. La serie ”o” (o1, o3) integra tecniche di ragionamento

come il *chain-of-thought* durante l’inferenza, migliorando le prestazioni in compiti scientifici e logico-matematici [50]. L’erogazione è vincolata ad API cloud proprietarie; in ambito enterprise è possibile operare tramite servizi dedicati (es. Azure OpenAI Service) che offrono garanzie contrattuali di isolamento dei dati.

Claude 4 (Anthropic). I modelli Claude adottano approcci come la *Constitutional AI* [2], un protocollo di addestramento guidato da principi per migliorare l’allineamento. Anche questi modelli sono erogati in modalità cloud; Anthropic propone modalità contrattuali (ad esempio *Zero Data Retention*) per rispondere a esigenze enterprise [38].

Gemini (Google DeepMind). La famiglia Gemini [43] è caratterizzata da multimodalità nativa e da finestre di contesto molto estese (fino a un milione di token per Gemini 3 Pro). Anche in questo caso l’accesso è fornito tramite servizi cloud.

1.2.2 Motivazioni per l’adozione di modelli locali

Nel caso in esame, le prestazioni assolute non sono l’unico criterio di scelta. Il sistema descritto in questa tesi deve poter operare su dati aziendali interni e deve quindi privilegiare controllo, tracciabilità e assenza di dipendenze da servizi esterni. Per questo l’architettura utilizza modelli eseguiti interamente in locale (*self-hosted*). La scelta si fonda su tre motivazioni, sintetizzate nella Tabella 1.3.

Tabella 1.3: Motivazioni per l’adozione di modelli LLM on-premise.

Motivazione	Descrizione	Riferimento normativo / letteratura
Sovranità del dato e conformità	L’esecuzione locale evita la trasmissione di dati verso infrastrutture di terze parti, anche in presenza di garanzie contrattuali (ZDR, DPA).	GDPR [9]; rischi del cloud nella gestione di dati sensibili.
Costi operativi nel lungo periodo	Il pricing a consumo dei provider cloud può risultare più oneroso rispetto all’ammortamento dell’hardware locale in scenari ad alto volume.	[3]; rischio di <i>vendor lock-in</i> .
Controllo e personalizzazione	I modelli open-weight (LLaMA, Mistral, Qwen) consentono fine-tuning supervisionato e allineamento su dati propri, senza le restrizioni delle policy dei sistemi proprietari.	[31, 18, 28]

Per queste ragioni, i modelli proprietari sono mantenuti come riferimento comparativo, mentre l’analisi progettuale si concentra sulle soluzioni eseguibili localmente. Tale impostazione rende coerente la scelta del modello con i vincoli di privacy, costo e governo dell’infrastruttura discussi nei capitoli successivi.

1.2.3 Modelli open-weight per uso on-premise

Llama 4 (Meta AI). Rilasciata nell’aprile 2025, la famiglia Llama 4 [47] adotta architetture *Mixture of Experts* (MoE) e supporta la multimodalità. Le varianti *Scout* e *Maverick* bilanciano parametri effettivi e numero di esperti per gestire contesti estesi fino a 10 milioni di token. La licenza open-weight presenta clausole specifiche da valutare in fase di adozione, in particolare per operatori e per l’Unione Europea.

Mistral 3 (Mistral AI). Nel 2025 Mistral AI ha ampliato la propria offerta con modelli orientati all’integrazione locale [48]. Il modello di punta, *Mistral Large 3*, è un modello MoE multilingue; la famiglia include varianti ottimizzate per l’esecuzione su hardware locale. La distribuzione sotto licenza Apache 2.0 facilita l’impiego in progetti aperti.

Qwen3 (Alibaba Cloud). La generazione Qwen3 [57] introduce modalità operative differenziate per task logici ed efficienza. La famiglia copre modelli da dimensioni molto ridotte fino a varianti MoE con finestre di contesto estese. Nel Capitolo 3 si descrive la scelta della variante 8B per il sistema proposto, motivata dal bilancio tra qualità semantica e velocità di inferenza su CPU-only.

1.2.4 Quantizzazione e inferenza on-premise

L’esecuzione locale di un LLM richiede un ulteriore passaggio: ridurre il costo computazionale dell’inferenza senza compromettere eccessivamente la qualità. La tecnica più rilevante in questo senso è la quantizzazione, cioè la riduzione della precisione numerica dei pesi del modello da FP32 o FP16/BF16 a rappresentazioni a bassa precisione, tipicamente a 8 o 4 bit. La Tabella 1.4 riassume l’impatto delle principali tecniche su memoria e qualità per un modello da 7 miliardi di parametri.

Tabella 1.4: Impatto della quantizzazione su occupazione di memoria e qualità per un modello da 7 miliardi di parametri. I valori sono indicativi e tratti da [7, 11].

Formato	Bit/peso	VRAM (7B)	Degr. qualità	Tecnica
FP32	32	~28 GB	baseline (0%)	Standard
FP16/BF16	16	~14 GB	≈ 0%	Standard
GPTQ	8	~8 GB	<1%	[11]
GGUF Q4_K_M	4	~4 GB	1–3%	llama.cpp
NF4 (QLoRA)	4	~4 GB	<1%	[7]

Nel progetto questa dimensione è particolarmente importante, perché il modello deve convivere con database, servizi applicativi e pipeline RAG nello stesso ambiente containerizzato. L’impatto della quantizzazione sulla qualità della generazione in contesti specifici è analizzato nel Capitolo 4.

Un ambito collegato è la compressione della *KV Cache*, struttura che memorizza i tensori Key e Value durante l’inferenza autoregressiva. Poiché la sua dimensione cresce linearmente con la lunghezza del contesto, la KV Cache può diventare il principale collo di bottiglia nei modelli a lungo contesto. In questa direzione si colloca TurboQuant [44], framework di compressione *data-oblivious* presentato da Google Research nel 2026. Il metodo combina PolarQuant, che normalizza la distribuzione dei dati tramite proiezione in coordinate polari, e Quantized Johnson-Lindenstrauss (QJL),

che limita l'errore introdotto dalla quantizzazione. Sebbene questa tecnica non sia parte dell'implementazione descritta nella tesi, è rilevante perché mostra una direzione concreta per estendere in futuro la finestra di contesto dei modelli locali senza aumentare proporzionalmente il consumo di memoria.

1.2.5 Estensione del contesto: Infini-attention

Mentre la compressione della KV Cache riduce il *footprint* di memoria a parità di contesto, altre architetture intervengono direttamente sul meccanismo di attenzione. Nell'aprile 2024, Munkhdalai et al. hanno presentato **Infini-attention** [25], una tecnica che combina attenzione locale mascherata e memoria compressiva a lungo termine. La prima mantiene alta risoluzione sulle relazioni immediate; la seconda conserva una sintesi del contesto passato con costo di memoria limitato. Questo approccio è di interesse per i sistemi RAG perché apre alla gestione di corpora documentali più ampi, pur rimanendo compatibile con scenari di inferenza locale.

1.2.6 Ollama: server di inferenza locale

Alla luce dei vincoli descritti, il progetto adotta un'infrastruttura di inferenza locale gestita tramite **Ollama** [49], un server open source che espone modelli locali in formato GGUF tramite un'API REST compatibile con la specifica *OpenAI Chat Completions*. Ollama semplifica il ciclo di vita dei modelli locali: l'installazione avviene tramite un comando semplice (`ollama pull <modello>`), la gestione del contesto e del formato di prompt è automatizzata, e il server può essere distribuito come container Docker. Nel contesto di questo progetto, Ollama serve sia il modello di chat sia il modello di embedding (`omic-embed-text`), il cui ruolo è descritto nella Sezione 1.3.3.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Motivazione e definizione

Il paradigma *Retrieval-Augmented Generation* (RAG), formalizzato da Lewis et al. [20], risponde al limite degli LLM come sistemi a conoscenza chiusa (*closed-book*). In un modello generativo tradizionale, infatti, la conoscenza disponibile è codificata nei parametri e non può essere aggiornata senza un nuovo ciclo di addestramento o fine-tuning.

In un sistema RAG, la generazione viene preceduta da una fase di *retrieval* dinamico. Prima di invocare l'LLM, il sistema interroga una base di conoscenza esterna e recupera i documenti più pertinenti rispetto alla query. Tali documenti vengono poi inseriti nel prompt (*prompt augmentation*), così che la risposta sia fondata non solo sulla conoscenza parametrica del modello, ma anche su fonti aggiornate e verificabili [30]. Questa separazione tra modello e conoscenza esterna è il motivo per cui RAG si adatta bene a domini aziendali in evoluzione.

1.3.2 Architettura del sistema RAG

Dal punto di vista architetturale, un sistema RAG separa il momento in cui la conoscenza viene preparata dal momento in cui viene interrogata. Ne derivano due pipeline distinte, messe a confronto nella Tabella 1.5 e illustrate nella Figura 1.1: la pipeline di **ingestione**, eseguita offline, e la pipeline di **inferenza**, attivata in tempo reale a ogni richiesta utente.

Tabella 1.5: Le due pipeline del sistema RAG a confronto.

Fase	Ingestion pipeline (offline)	Query pipeline (real-time)
1	Pre-processing e chunking dei documenti sorgente	Embedding della query utente nello stesso spazio vettoriale dei documenti
2	Generazione degli embedding tramite modello locale (nomic-embed-text)	Ricerca ANN nel database vettoriale (<i>top-k</i>) con filtri su metadati
3	Indicizzazione in Qdrant con payload di metadati strutturati	Augmented generation: contesto recuperato + query → LLM
<i>Trigger</i>	Esecuzione manuale o schedulata (es. giornaliera)	Ad ogni richiesta utente
<i>Latenza</i>	Minuti / ore (elaborazione batch)	Millisecondi (inferenza real-time)

Pipeline di ingestione

La pipeline di ingestione trasforma i documenti grezzi in una rappresentazione adatta alla ricerca semantica. Il risultato non è più soltanto un archivio di testi, ma un indice vettoriale interrogabile per similarità. Le fasi principali sono le seguenti:

- **Pre-processing e chunking.** Il documento viene suddiviso in unità testuali (*chunk*) di dimensione controllata. La strategia di chunking influisce direttamente sulla qualità del retrieval: chunk troppo piccoli perdono contesto, mentre chunk troppo grandi possono diluire il segnale semantico e superare la finestra di contesto del modello di embedding [6].
- **Generazione degli embedding.** Ogni chunk viene trasformato in un vettore denso da un modello di embedding $f: \mathcal{T} \rightarrow \mathbb{R}^d$, in modo che documenti semanticamente simili producano vettori vicini secondo la similarità coseno:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \quad (1.3)$$

- **Indicizzazione nel database vettoriale.** I vettori vengono persistiti in **Qdrant** [55], che implementa l'algoritmo HNSW (*Hierarchical Navigable Small World*) [22] per la ricerca approssimata (*Approximate Nearest Neighbor*, ANN). La scelta di Qdrant rispetto ad alternative quali pgvector, Chroma e Milvus è motivata nel Capitolo 2.

Pipeline di inferenza

La pipeline di inferenza utilizza l'indice costruito offline per rispondere a una query utente. A differenza dell'ingestione, questa fase è vincolata dalla latenza percepita e deve quindi bilanciare qualità del recupero, numero di documenti restituiti e costo della generazione.

- **Embedding della query.** La query dell'utente viene proiettata nello stesso spazio vettoriale dei documenti indicizzati con lo stesso modello di embedding.

- **Retrieval.** Si esegue una ricerca ANN in Qdrant per recuperare i k documenti più simili alla query. La ricerca può essere limitata tramite *metadata filtering* a sottoinsiemi coerenti (ad esempio, documenti appartenenti a una specifica epoca temporale).
- **Augmented generation.** Il contesto recuperato viene concatenato alla query nel prompt inviato all'LLM. Il modello genera la risposta basandosi sulla propria conoscenza parametrica e sulle informazioni esterne fornite.

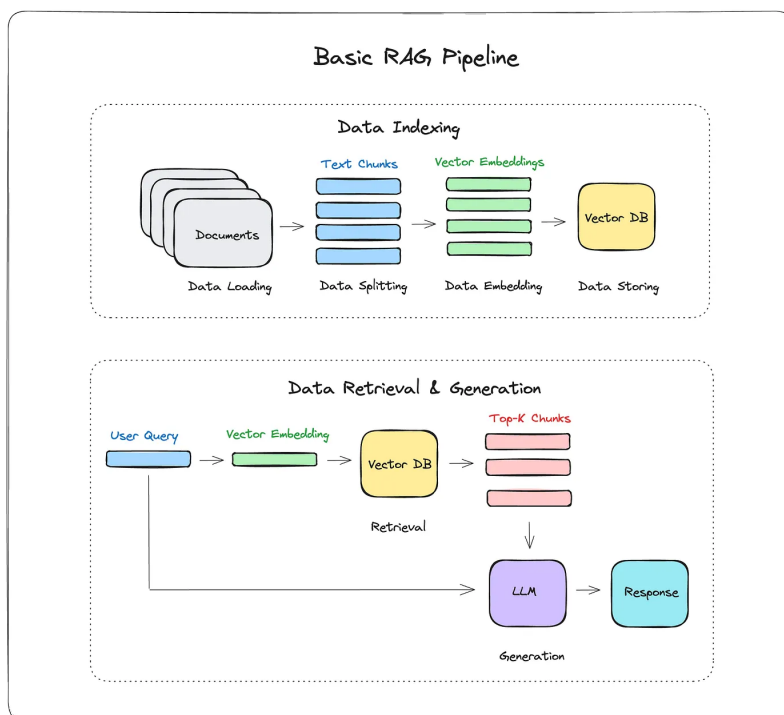


Figura 1.1: Schema della pipeline RAG: la sezione superiore rappresenta la fase di ingestione offline; la sezione inferiore la pipeline di inferenza in tempo reale [20].

1.3.3 Modelli di embedding

La qualità del retrieval dipende in larga misura dal modello di embedding: se la rappresentazione vettoriale non preserva correttamente il significato dei documenti, anche il modello generativo riceverà un contesto poco pertinente. Il vincolo di esecuzione on-premise restringe quindi la scelta a modelli distribuibili localmente tramite Ollama. La valutazione comparativa è stata condotta su quattro candidati — `nomic-embed-text`, `mxbai-embed-large`, `all-minilm` e `multilingual-e5-large` — secondo criteri di qualità per la lingua italiana, occupazione di memoria, compatibilità con Ollama e punteggi sul benchmark MTEB (*Massive Text Embedding Benchmark*) [24]. L'analisi completa e la motivazione della scelta finale di `nomic-embed-text` [26] sono presentate nel Capitolo 2.

1.3.4 Database vettoriali

Un database vettoriale è un sistema di storage specializzato per vettori densi ad alta dimensionalità, progettato per rispondere a query di tipo *k-nearest neighbor* (kNN). A differenza dei database relazionali, dove le query sono esatte, la ricerca vettoriale è per natura approssimata: si cercano i vettori più vicini accettando un trade-off tra richiamo (*recall*) e velocità (*throughput*).

L'algoritmo HNSW costruisce un grafo gerarchico multi-livello nello spazio vettoriale, con complessità di ricerca approssimativamente $O(\log n)$ rispetto al numero di punti n — contro $O(n)$ della ricerca lineare esatta [22] — garantendo scalabilità anche per ampi *corpora*.

1.3.5 Agentic RAG

Il RAG lineare descritto finora segue un flusso deterministico: recupero dei documenti, costruzione del prompt e generazione della risposta. Nei casi in cui la query richiede più passaggi logici, confronto tra fonti o riformulazioni successive, tale schema può risultare insufficiente. L'*Agentic RAG* estende questo approccio utilizzando l'LLM come unità di ragionamento capace di invocare strumenti, pianificare ricerche successive e valutare la pertinenza dei documenti recuperati. Architetture come ReAct [35] formalizzano questo ciclo combinando ragionamento verbale e azioni.

In contesti RAG avanzati, tecniche come Self-RAG [1] e Corrective RAG (CRAG) [34] introducono meccanismi di controllo sulla qualità del retrieval e della risposta generata. Il sistema può così rilevare lacune informative, richiedere nuovi documenti o modificare il prompt prima di produrre la risposta finale. Le rassegne più recenti evidenziano che questa evoluzione è particolarmente utile quando le query richiedono sintesi da fonti eterogenee o passaggi intermedi di ragionamento [12].

Un'altra direzione di sviluppo è rappresentata da **GraphRAG**, formalizzato da Microsoft Research nel 2024 [8]. A differenza del RAG vettoriale classico, che è efficace nel recupero di passaggi localmente simili alla query ma meno adatto a interrogazioni globali sull'intero dataset, GraphRAG estrae entità e relazioni dai testi sorgente e costruisce un grafo di conoscenza gerarchico. In fase di retrieval, il sistema può quindi aggregare riassunti semantici di intere *community* di concetti, ottenendo un livello di astrazione più adeguato all'analisi di *corpora* documentali estesi.

1.3.6 Tecniche di adattamento al dominio

Il collegamento tra LLM e conoscenza aziendale può essere realizzato in modi diversi. RAG mantiene separati i pesi del modello e il corpus documentale; il fine-tuning, invece, modifica il modello affinché si adatti a un dominio o a uno stile di risposta specifico. Questa sezione introduce le principali tecniche di adattamento a parametri ridotti, che costituiscono il riferimento teorico per il confronto discusso nel Capitolo 4.

Fine-tuning con adattatori: LoRA

Il fine-tuning completo di un LLM richiede risorse computazionali elevate. Hu et al. [14] hanno proposto *Low-Rank Adaptation* (LoRA), che vincola gli aggiornamenti dei pesi a matrici di rango basso. Dato un layer lineare con matrice $\mathbf{W}_0 \in \mathbb{R}^{m \times n}$, LoRA introduce:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA} \quad (1.4)$$

con $\mathbf{A} \in \mathbb{R}^{r \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times r}$ e $r \ll \min(m, n)$. I pesi originali $\mathbf{W}_0$ rimangono congelati; solo $\mathbf{A}$ e $\mathbf{B}$ vengono ottimizzati. Con $r = 8$ e $m = n = 4096$, il numero di parametri addestrabili si riduce di circa due ordini di grandezza rispetto al fine-tuning completo.

QLoRA: fine-tuning a 4-bit

Quantized LoRA (QLoRA) [7] combina la quantizzazione a 4-bit del modello base con adattatori LoRA a precisione piena. I pesi vengono quantizzati in formato NF4 (*NormalFloat4*), ottimizzato per le distribuzioni tipiche dei pesi neurali; durante il forward pass i pesi vengono de-quantizzati dinamicamente in BF16, mentre gli adattatori operano in BF16.

La Tabella 1.6 riassume i principali trade-off tra le due varianti.

Tabella 1.6: Confronto tra LoRA e QLoRA per un modello base da 7B parametri.

Caratteristica	LoRA	QLoRA
Precisione pesi base	FP16/BF16	NF4 (4-bit)
Precisione adattatori	FP16/BF16	BF16
Memoria GPU (modello 7B)	~14 GB	~4 GB
Overhead de-quantizzazione	assente	presente
Degradazione qualitativa stimata	trascurabile	<1% su benchmark principali

DoRA: Weight-Decomposed Low-Rank Adaptation

Nel 2024 Liu et al. hanno introdotto **DoRA** (*Weight-Decomposed Low-Rank Adaptation*) [21]. Il metodo parte dall'osservazione che LoRA approssima bene l'aggiornamento direzionale dei pesi, ma cattura con minore efficacia le variazioni di magnitudine osservate nel fine-tuning completo. DoRA separa quindi il tensore dei pesi in una componente di magnitudine e una componente direzionale: la prima viene aggiornata direttamente, mentre la seconda sfrutta l'approssimazione a rango basso di LoRA. In questo modo si riduce il divario prestazionale rispetto al fine-tuning completo, mantenendo un numero contenuto di parametri addestrabili.

Prompt engineering e inference-time intervention

In alternativa al fine-tuning parametrico, è possibile adattare il comportamento di un LLM intervenendo sulla formulazione dell'input senza modificare i pesi. Tecniche come *few-shot prompting* [4] e *chain-of-thought prompting* [33] guidano il modello verso risposte più strutturate, arricchendo il prompt con esempi o istruzioni di ragionamento. Nel sistema descritto in questa tesi, il prompt engineering è impiegato nella definizione del *system prompt* e nell'iniezione strutturata del contesto RAG; le strategie adottate e il loro impatto sulla qualità delle risposte sono analizzati nel Capitolo 4.

1.4 Framework di orchestrazione: Haystack

La costruzione di un sistema RAG non consiste soltanto nella scelta del modello generativo. È necessario coordinare modelli di embedding, database vettoriali, LLM e fasi di pre-processing attraverso interfacce coerenti e componibili. **Haystack** [40], sviluppato da deepset, è un framework Python open source progettato per organizzare questi componenti in pipeline esplicite.

Nella versione 2.x adottata in questo progetto, ogni pipeline è un grafo diretto aciclico (DAG) di componenti con interfacce tipizzate, collegabili in modo dichiarativo. Sono disponibili integrazioni native con Qdrant (`QdrantDocumentStore`), Ollama (`OllamaDocumentEmbedder`, `OllamaTextEmbedder`, `OllamaGenerator`) e le principali operazioni di pre-processing (`DocumentCleaner`, `DocumentSplitter`). Il dettaglio implementativo della pipeline di ingestione e di retrieval è descritto nel Capitolo 3.

1.5 Sintesi e posizionamento del lavoro

Il presente capitolo ha fornito il quadro teorico su cui si fonda il sistema presentato nei capitoli successivi. Le architetture Transformer costituiscono il substrato computazionale comune a componenti di generazione e embedding; i loro limiti in contesti a conoscenza chiusa — conoscenza statica, allucinazioni, privacy — motivano l'adozione del paradigma RAG. La scelta di operare con modelli e infrastrutture interamente on-premise (Ollama, Qdrant, Haystack) risponde ai vincoli di sovranità del dato e alla sostenibilità computazionale identificata nell'analisi dei requisiti. Le tecniche di fine-tuning a parametri ridotti (LoRA, QLoRA) costituiscono il termine di confronto teorico per la valutazione delle strategie di adattamento al dominio nel Capitolo 4.

Capitolo 2

Analisi dei Requisiti e Progettazione del Sistema

Il capitolo traduce il quadro teorico introdotto in precedenza in un problema progettuale concreto. L'obiettivo non è costruire un sistema RAG generico, ma integrare un assistente nel contesto operativo di Intré, rispettando i vincoli organizzativi, infrastrutturali e di riservatezza già presenti. Per questo la trattazione parte dal modello aziendale delle Gilde, identifica il caso d'uso e il corpus informativo disponibile, formalizza i requisiti e motiva infine le scelte tecnologiche che portano alla proposta architettuale.

2.1 Il contesto aziendale: Intré S.r.l.

Intré S.r.l.¹ è una software factory italiana con sede a Monza. L'azienda sviluppa prodotti digitali per settori eterogenei, tra cui energia, assicurazioni, automotive ed entertainment, e organizza il lavoro in team Scrum cross-funzionali. La dimensione aziendale, pari a circa settanta persone, consente di mantenere processi interni formalizzati senza rinunciare a una circolazione diretta della conoscenza tra i team [15].

Il payoff *Learn / Code / Deploy Value* esplicita l'ordine con cui Intré interpreta il proprio modo di produrre valore: prima l'apprendimento, poi la realizzazione del software, infine il rilascio di valore al cliente. Questa impostazione è coerente con il concetto di *Learning Organization* proposto da Peter Senge in *The Fifth Discipline* [29]. Nel modello di Senge l'apprendimento organizzativo non coincide con la sola formazione individuale, ma richiede padronanza personale, modelli mentali esplicitati, visione condivisa, apprendimento di gruppo e pensiero sistemico. La quinta disciplina, cioè il pensiero sistemico, collega le altre quattro e permette di leggere l'organizzazione come un insieme di relazioni, feedback e dipendenze, non come una somma di attività isolate.

Nel caso di Intré questa prospettiva non rimane un principio astratto. Il modello aziendale prevede un insieme di pratiche dedicate all'apprendimento continuo: Gilde, Camp, Community, Skill Matrix, certificazioni, Academy, Community of Practice per speaker, obiettivi di apprendimento, formazione in aula e corsi di lingua [15]. Tali pratiche rendono l'apprendimento una responsabilità distribuita, sostenuta da tempo, budget e momenti di restituzione pubblica. La certificazione

¹<https://www.intre.it>

ISO 27001 e le competenze maturate in ambiti come cloud-native, cybersecurity e intelligenza artificiale rendono inoltre particolarmente rilevante il tema della gestione sicura della conoscenza interna.

2.1.1 Il modello delle Gilde

Le **Gilde** (*gilde*) sono uno degli strumenti principali con cui Intré rende operativa la propria idea di apprendimento organizzativo. Sono cicli rapidi di apprendimento di gruppo, della durata di quattro mesi, costruiti a partire da temi proposti direttamente dalle persone dell'azienda [15, 16]. Una Gilda nasce quando una proposta raccoglie un numero minimo di partecipanti; il gruppo lavora in autogestione e dedica al tema scelto una quota stabile del proprio tempo lavorativo. L'esito non è un esame o una valutazione formale, ma un risultato di apprendimento da presentare al Camp successivo. Il modello combina quindi autonomia nella scelta dei temi, durata definita, lavoro di gruppo e produzione di un risultato condivisibile. Le sue caratteristiche principali sono:

- **Formazione spontanea:** ogni persona può proporre o aderire a una Gilda su qualsiasi argomento, tecnico o non tecnico (es. linguaggi di programmazione, ceramica, game development, intelligenza artificiale).
- **Ciclo temporale (Epoch):** le Gilde si ricreano ogni quattro mesi; ogni ciclo costituisce un'“epoca” distinta, con nuovi temi e nuovi gruppi. Questa scansione periodica evita che l'apprendimento rimanga legato a iniziative isolate e lo inserisce in una routine aziendale riconoscibile.
- **Output condivisibile:** al termine del periodo la Gilda produce un risultato tangibile (prototipo, report, repository, presentazione) che viene archiviato e condiviso. La restituzione pubblica durante il Camp trasforma l'esperienza del gruppo in conoscenza disponibile anche per il resto dell'organizzazione.
- **Ampiezza tematica:** dall'archivio pubblico del sito aziendale emergono Gilde su temi come *Angular*, *F#*, *IA agentiva*, *game development* con Phaser, ceramica, selezione fotografica con AI, e molti altri [16].

Il ciclo di vita delle Gilde è gestito tramite **i3Guild**, una piattaforma web interna sviluppata con Next.js, PostgreSQL e Keycloak. Questa applicazione conserva nel tempo proposte, descrizioni, partecipanti e risultati delle Gilde. Di conseguenza, i3Guild non è soltanto uno strumento amministrativo: è una memoria organizzativa parziale, nella quale si depositano interessi, competenze, esperimenti e risultati prodotti nel tempo. Proprio per questo rappresenta sia il contesto applicativo di integrazione sia la principale sorgente dati del sistema RAG progettato in questa tesi.

2.2 Caso d'uso: Proposta di nuove Gilde Intré

Il caso d'uso individuato riguarda il supporto alla proposta di nuove Gilde. Quando una persona vuole avviare una nuova iniziativa, può essere utile conoscere esperienze precedenti, temi già affrontati, risultati ottenuti e partecipanti coinvolti. Oggi queste informazioni esistono, ma sono distribuite tra database e contenuti editoriali. Il sistema proposto mira a renderle consultabili tramite query in linguaggio naturale, generando risposte contestualizzate e ancorate a fonti interne. A tale

scopo viene adottato il paradigma *Retrieval-Augmented Generation* (RAG), introdotto da Lewis et al. [19] per estendere la memoria non parametrica dei Large Language Model (LLM) tramite recupero dinamico di documenti da un corpus esterno.

2.2.1 Motivazione del caso d'uso

Il corpus delle Gilde archiviate in i3Guild, insieme ai relativi blog post, costituisce una base di conoscenza aziendale di valore: raccoglie descrizioni di progetti, obiettivi, rischi, risultati raggiunti e annotazioni dei partecipanti, accumulate in oltre trenta epoch a partire dal 2016 [16]. Tuttavia questa conoscenza è frammentata. Una parte risiede in un database relazionale, ottimo come sorgente applicativa ma poco accessibile tramite domande in linguaggio naturale; un'altra parte è distribuita in post e contenuti non sempre facili da reperire. Un assistente RAG consente di trasformare tale patrimonio in uno strumento operativo: riduce il tempo di ricerca, facilita il riuso delle esperienze pregresse e rende più trasferibile la conoscenza tra i team.

2.2.2 Obiettivi del caso d'uso

- Fornire risposte affidabili e tracciabili su contenuti presenti nei repository aziendali, riducendo il rischio di allucinazioni tipico della generazione puramente parametrica [19].
- Ridurre il tempo di ricerca delle informazioni per attività operative e decisionali interne a Intré.
- Consentire l'esecuzione del servizio in ambiente on-premise per requisiti di sovranità del dato, in conformità al Regolamento Generale sulla Protezione dei Dati (GDPR) [10].
- Integrare il sistema nel flusso i3Guild esistente, minimizzando l'impatto sul processo operativo consolidato.

2.2.3 Descrizione dei dati e analisi del corpus

Per progettare correttamente la pipeline di ingestione è necessario chiarire quali informazioni debbano alimentare l'indice. I dati di ingresso al sistema RAG comprendono tre categorie:

- **Documenti delle Gilde:** specifiche di progetto, obiettivi, rischi, risultati attesi e raggiunti, annotati con metadati strutturati (epoche, partecipanti, modalità di partecipazione).
- **Blog post:** post pubblicati dai partecipanti alle gilde che ne sintetizzano i risultati e le esperienze maturate.
- **Documenti non strutturati:** file PDF, HTML e note operative con metadati associati.

Dall'analisi del corpus i3Guild, estratto il 15/02/2026, emergono le statistiche riportate nelle Tabelle 2.1 e 2.2. Questi valori non hanno solo funzione descrittiva: determinano la granularità dell'indicizzazione, la strategia di chunking e la scelta del modello di embedding.

La somma media dei campi testuali rilevanti (`summary + goals + outcome + risks + achievedResultsDescription`) ammonta a circa 1.900 caratteri, corrispondenti a circa 400 token. Anche nel caso peggiore la somma non supera i 7.000–8.000 token, rimanendo compatibile con la finestra di contesto del modello di embedding selezionato (Sezione 2.4.3). Da ciò deriva una conseguenza progettuale importante:

Tabella 2.1: Statistiche delle entità principali del corpus i3Guild (estrazione 15/02/2026).

Entità	Conteggio	Note
Gilda	270	Gestibile per re-indicizzazioni complete
Epoch	30	≈ 9 gilde per epoca in media
Relazioni Utente-Gilda	1.166	$\approx 4,3$ partecipanti medi per gilda
Utenti esterni	3	Trascurabile ai fini del RAG

Tabella 2.2: Statistiche dei campi testuali delle entità Gilda (valori in caratteri).

Campo	Media	Min	Max	Vuoti	Note
summary	610	0	18.224	5%	Campo più ricco; outlier rilevante
goals	258	0	2.698	35%	Spesso breve o assente
outcome	133	0	2.145	36%	Spesso breve o assente
risks	106	0	1.245	38%	Spesso breve o assente
achievedResultsDescription	782	0	6.421	74%	Lungo quando presente
journal	≈ 1	1	1	99%	Praticamente inutilizzato; escluso

nella maggior parte dei casi una Gilda può essere trattata come documento consolidato, senza frammentare aggressivamente i campi testuali e senza perdere il contesto semantico che lega obiettivi, rischi e risultati.

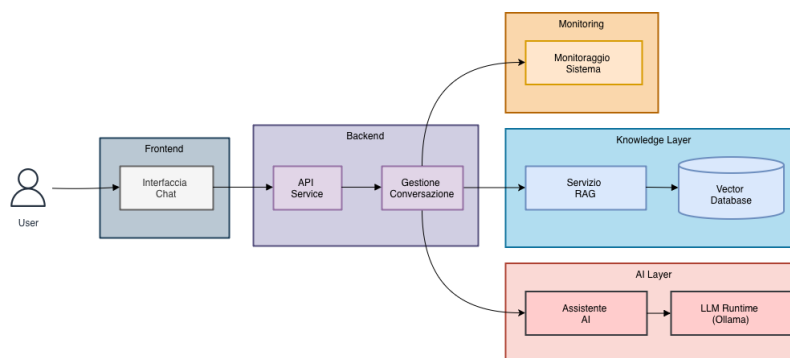


Figura 2.1: Diagramma del caso d'uso “Intré”: attori, interazioni principali e confini del sistema.

2.3 Requisiti di sistema

Il caso d'uso appena descritto permette di passare dalla motivazione generale ai requisiti del sistema. Essi sono stati ricavati dall'analisi del corpus, dai vincoli infrastrutturali di Intré e dalle policy aziendali in materia di protezione dei dati. La Tabella 2.3 sintetizza i requisiti ad alto livello, suddivisi per categoria.

Tabella 2.3: Riepilogo requisiti ad alto livello.

Categoria	Requisiti principali
Funzionali	Ricerca semantica full-text su documenti interni; generazione di risposte contestualizzate con riferimenti alle fonti; capacità di filtraggio per metadati (epoca, modalità di partecipazione, proprietario della gilda).
Privacy e conformità	Esecuzione on-premise; nessun invio di dati a servizi cloud esterni; logging e auditing degli accessi; protezione dei dati in conformità al GDPR [10]. I rischi relativi alla privacy nei sistemi RAG, tra cui la possibile estrazione dei dati di retrieval, sono discussi in letteratura [36]; la scelta on-premise costituisce la principale misura mitigativa adottata.
Performance	Latenza di risposta nell'ordine del secondo per query semplici; throughput scalabile per carichi concorrenti moderati; inferenza CPU-only come scenario primario.
Costi e sostenibilità	Minimizzazione dei costi operativi tramite uso efficiente delle risorse (quantizzazione dei modelli, inferenza CPU); adozione di soluzioni open-source senza costi per licenze.
Manutenibilità	Componenti modulari con interfacce standardizzate; automazione della pipeline di ingestione; documentazione e test automatizzati.

2.3.1 Vincoli architetturali

I requisiti precedenti si traducono in alcuni vincoli architetturali non negoziabili, che orientano direttamente la scelta dei componenti:

- **On-premise obbligatorio:** i dati delle Gilde, incluse informazioni sui partecipanti e sui progetti, non possono transitare su infrastrutture cloud di terze parti. Il deployment locale consente di soddisfare il principio di integrità e riservatezza previsto dal GDPR [10].
- **Compatibilità con lo stack i3Guild:** l'integrazione deve essere coerente con l'infrastruttura esistente (Docker Compose, PostgreSQL/Prisma, Next.js, Keycloak per l'autenticazione).
- **Limitazioni hardware:** l'ambiente Docker dispone di 8 GB di RAM complessivi, che dovranno essere condivisi tra il modello di embedding e il modello di generazione del testo. Questo vincolo è determinante per la selezione del modello LLM (Sezione 2.4.4).
- **No API esterne per LLM ed embedding:** tutti i modelli devono essere eseguiti localmente tramite Ollama, eliminando dipendenze da provider come OpenAI o Anthropic.

2.4 Scelte tecnologiche: analisi comparativa

Le scelte tecnologiche sono state effettuate a partire dai vincoli appena definiti. Per ciascun componente principale dello stack sono state valutate alternative compatibili con l'esecuzione on-premise, privilegiando soluzioni semplici da integrare nello stack i3Guild, osservabili e sostenibili rispetto alle risorse disponibili.

2.4.1 Framework di orchestrazione RAG

Il framework di orchestrazione deve collegare ingestione, embedding, retrieval e generazione senza introdurre eccessiva complessità operativa. Sono stati considerati tre framework open-source: **LangChain**, **LlamaIndex** e **Haystack 2.x**.

LangChain offre l'ecosistema più ampio di integrazioni e una forte vocazione verso workflow agentici e *multi-step reasoning*. Nel 2024 l'introduzione di **LangGraph** [46] ha ampliato questa impostazione, permettendo di modellare applicazioni multi-agente come grafi di stato. Tale flessibilità è utile in scenari complessi, ma nel progetto in esame introduce un costo non trascurabile in termini di curva di apprendimento, stabilità delle API e manutenzione, anche alla luce dei *breaking changes* frequenti riportati in letteratura [52].

LlamaIndex è orientato alla gestione dei dati e al retrieval avanzato. Offre numerosi data connector e strutture di indicizzazione specializzate, risultando particolarmente adatto a pipeline di question-answering documentale [39]. Rispetto al caso considerato, tuttavia, l'obiettivo principale non è solo indicizzare sorgenti eterogenee, ma integrare una pipeline osservabile e facilmente testabile nello stack applicativo esistente.

Haystack 2.x (deepset) adotta un modello a componenti tipizzati con input/output espliciti, organizzati in pipeline dichiarative serializzabili in YAML. Studi comparativi evidenziano che Haystack offre il minore overhead di orchestrazione (≈ 5.9 ms) e il minore consumo di token rispetto a LangChain (≈ 10 ms) su benchmark standardizzati [37]. La sua architettura pipeline-first favorisce la tracciabilità, i test e la sostituzione dei componenti in produzione [51].

Tabella 2.4: Confronto sintetico dei framework RAG candidati.

Criteria		LangChain	LlamaIndex	Haystack 2.x
Filosofia		Agente/imperativa	Data-first	Pipeline dichiarativa
Overhead	orche-	≈ 10 ms	≈ 6 ms	≈ 5.9 ms
Integrazione		Nativa	Nativa	Nativa
Qdrant				
Integrazione	Olla-	Sì	Sì	Sì
ma				
Testabilità pipeline		Media	Media	Alta
Serializzazione		No	Parziale	Sì
YAML				
Adatto per produ-		Sì (con overhead)	Sì	Sì (preferito)
zione RAG				

Scelta motivata: Haystack 2.x. L'architettura a componenti tipizzati, la documentazione rigorosa, le integrazioni native con Qdrant (`QdrantDocumentStore`) e Ollama (`OllamaDocumentEmbedder`, `OllamaTextEmbedder`), il minore overhead di orchestrazione e la serializzabilità delle pipeline la rendono la scelta più adeguata per un sistema RAG production-oriented in ambiente on-premise [52].

2.4.2 Database vettoriale

La ricerca di similarità approssimata (*Approximate Nearest Neighbor*, ANN) su vettori ad alta dimensionalità è il componente infrastrutturale che abilita il retrieval. La scelta del database vettoriale incide su latenza, filtraggio per metadati, semplicità di deployment e capacità di manutenzione dell'indice. Sono stati quindi valutati quattro database vettoriali self-hostable.

Tabella 2.5: Analisi comparativa dei database vettoriali candidati.

Database	Impl.	Indice	Filtering	Dashboard	Deployment
Qdrant	Rust	HNSW	Pre-filtering payload	Web UI integrata	1 container
pgvector	C	IVFFlat / HNSW	SQL WHERE	No (via PgAdmin)	Estensione PG
Milvus	Go/C++	HNSW, IVF	Attribute-based	Attu (separato)	Multi-container
Chroma	Python	HNSW (hnswlib)	Metadata base	No	1 container

Il confronto non si limita alle prestazioni pure: nel contesto i3Guild hanno peso anche la semplicità operativa e la possibilità di applicare filtri sui metadati delle Gilde. I principali criteri di esclusione e selezione sono i seguenti:

- **pgvector** è stato considerato in quanto l'infrastruttura i3Guild utilizza già PostgreSQL. Tuttavia, la letteratura segnala che le prestazioni di pgvector degradano significativamente oltre le centinaia di migliaia di vettori, con ritardi fino a 15 volte superiori rispetto a Qdrant in termini di throughput su dataset grandi [45]. Benchmark più recenti mostrano un divario più contenuto (Qdrant: ≈ 41 QPS a 99% recall su 50M vettori; pgvectorscale: ≈ 471 QPS), ma evidenziano che Qdrant ottiene latenze tail significativamente migliori (39% migliore a p95, 48% migliore a p99) [59]. Assenza di dashboard integrata e di payload filtering flessibile completano le ragioni per la non-selezione.
- **Milvus** è stato escluso per l'elevata complessità di deploy: richiede componenti aggiuntivi (`etcd`, `MinIO`), aumentando il carico operativo incompatibile con i vincoli del progetto.
- **Chroma** è implementato in Python e soffre di instabilità della persistenza dati nelle versioni precedenti; è più adatto a prototipi che a sistemi di produzione.
- **Qdrant** è scritto in Rust, garantisce prestazioni elevate e latenze stabili. La funzionalità di pre-filtering, che applica i filtri sui payload *prima* della ricerca vettoriale, è cruciale per query contestuali come “gilda dell'epoca X”: il filtering riduce lo spazio di ricerca prima del calcolo ANN, mantenendo il top- K accurato. Offre Web UI integrata su porta 6333, client TypeScript ufficiale compatibile con Next.js, e deploy in singolo container Docker [56].

Scelta motivata: Qdrant. Per le ragioni sopra esposte e in linea con il vincolo aziendale di self-hosting, Qdrant rappresenta il database vettoriale più adeguato al contesto del progetto.

2.4.3 Modello di embedding

Il modello di embedding converte i documenti testuali in vettori ad alta dimensionalità che ne codificano il significato semantico. Poiché il vincolo aziendale impone l'assenza di dipendenze da API esterne, tutti i modelli candidati devono essere serviti localmente tramite Ollama.

Tabella 2.6: Confronto dei modelli di embedding candidati per il sistema RAG.

Modello	Dim.	Peso	Italiano	Contesto (tok.)	Disponibilità Ollama
nomic-embed-text	768	275 MB	Buono	8.192	Nativa
mx-bai-embed-large	1.024	670 MB	Buono	512	Nativa
all-minilm	384	45 MB	Scarso	512	Nativa
multilingual-e5-large	1.024	1,3 GB	Ottimo	512	Import manuale

La selezione del modello è guidata da cinque criteri pesati: (1) qualità per la lingua italiana (peso 30%), in quanto i documenti delle Gilde sono interamente in italiano; (2) footprint di risorse (25%), essenziale in un ambiente Docker con risorse condivise; (3) compatibilità nativa con Ollama (20%); (4) dimensione dello spazio vettoriale (15%); (5) benchmark MTEB [23] (10%).

Scelta motivata: nomic-embed-text. Il modello, introdotto da Nussbaum et al. [26], è il primo modello open-source completamente riproducibile con finestra di contesto lunga (8.192 token) che supera **text-embedding-ada-002** di OpenAI sia sul benchmark MTEB che sul benchmark LoCo per contesto lungo. Con soli 137M parametri e un peso di 275 MB, offre il miglior rapporto qualità/footprint tra i candidati. La finestra di 8.192 token è determinante: permette di ingerire la quasi totalità dei documenti consolidati delle Gilde come singolo vettore senza chunking aggressivo, preservando il contesto semantico integrale. È rilasciato sotto licenza Apache 2.0, compatibile con l'utilizzo aziendale.

2.4.4 Server di inferenza LLM e selezione del modello di chat

Il componente di generazione del testo è gestito da **Ollama**, server di inferenza locale che supporta modelli in formato GGUF e consente l'uso di varianti quantizzate a 4 e 8 bit. La scelta del modello di chat è stata trattata come un problema sperimentale, non come una semplice preferenza tecnologica: sono stati testati dodici modelli disponibili nel registry Ollama per individuare il compromesso più adeguato tra qualità delle risposte, latenza e consumo di risorse nell'ambiente Docker vincolato a 8 GB di RAM totali.

Metodologia di valutazione

Il benchmark ha coinvolto dodici modelli su due categorie di query: *General* (comprensione del linguaggio, ragionamento, estrazione di entità, scrittura) e *RAG* (risposte basate su contesto recuperato). Per ogni query sono stati misurati: TTFT (*Time to First Token*), latenza totale, throughput in parole al secondo, picco di RAM e utilizzo medio della CPU. La qualità delle risposte è stata valutata su tre assi: accuratezza/ fedeltà ($Score_1$), precisione/pertinenza ($Score_2$) e coerenza ($Score_3$, valutata solo per query *General*).

Il punteggio globale (G) è calcolato come combinazione lineare pesata di quattro componenti normalizzate nell'intervallo $[0, 1]$:

$$G = 0.50 \cdot Q + 0.20 \cdot T + 0.15 \cdot (1 - R) + 0.15 \cdot (1 - C) \quad (2.1)$$

dove Q è la qualità media normalizzata, T è il throughput normalizzato, R è il consumo RAM normalizzato e C è il carico CPU normalizzato (con inversione per RAM e CPU perché valori più bassi sono preferibili).

Risultati del benchmark

Modello	Latenza (s)	Throughput (w/s)	RAM (MB)	CPU (%)	Qualità	G
deepscaler:latest	18.85	29.25	42.69	1.79	7.14	83.1
llama3.2:latest	8.94	21.43	37.26	1.78	7.50	76.7
qwen2.5:1.5b	35.04	30.67	42.98	3.41	6.45	76.7
gemma3:1b	6.74	28.01	42.83	7.05	7.74	69.7
llama3.2:3b	16.54	20.14	40.74	2.00	6.43	68.7
gemma3:latest	17.29	16.27	42.15	3.18	7.69	66.8
granite3.3:2b	10.35	20.58	41.82	5.11	7.76	66.8
phi4-mini:3.8b	8.78	19.90	82.20	1.91	7.31	66.3
lfn2.5-thinking:1.2b	14.16	13.55	43.52	2.41	7.59	65.1
ministral-3:3b	66.79	15.09	40.26	3.12	7.19	63.4
deepseek-r1:1.5b	19.26	14.55	42.66	2.26	6.60	61.9
phi4-mini-reasoning:latest	95.53	19.55	129.14	1.87	5.00	46.8

Motivazione della scelta: gemma3:1b

Alla luce dei risultati del benchmark e del vincolo di 8 GB di RAM disponibili per l'intero stack Docker, il modello selezionato è **gemma3:1b**. La scelta non coincide con il punteggio globale più alto in assoluto, ma con il miglior equilibrio rispetto allo scenario applicativo: risposte sufficientemente accurate, tempi compatibili con l'uso interattivo e ridotto impatto sugli altri container. Le motivazioni principali sono:

- **RAM compatibile con i vincoli:** il consumo medio di circa 43 MB di RAM è contenuto, lasciando margine adeguato per gli altri container (PostgreSQL, Qdrant, Next.js, RAG service).
- **Latenza più bassa tra i modelli con qualità elevata:** latenza media di 6.74 s, la più bassa tra i modelli con un punteggio di qualità ≥ 7.5 .
- **Qualità adeguata:** punteggio medio di qualità pari a 7.74, tra i più alti dell'intero benchmark.
- **Throughput elevato:** 28.01 parole/s, secondo valore più alto dopo qwen2.5:1.5b e deepscaler.

I modelli con punteggio globale superiore (**deepscaler**, **llama3.2:latest**) presentano rispettivamente latenze elevate (18.85 s) o qualità inferiore su task RAG. Il modello **phi4-mini:3.8b**, pur mostrando buona qualità, richiede circa il doppio della RAM rispetto agli altri (82 MB), riducendo il margine disponibile per l'intero stack. La scelta di **gemma3:1b** è da considerarsi adeguata per il prototipo attuale; in un ambiente con risorse RAM superiori, modelli più capaci (es. **granite3.3:2b** o **gemma3:latest**) potrebbero essere valutati come alternativa.

2.5 Proposta architetturale

Le scelte tecnologiche descritte convergono in una soluzione modulare e containerizzata. L'architettura separa responsabilità diverse – ingestione e normalizzazione dei dati, generazione degli embedding, persistenza vettoriale, retrieval e inferenza LLM – in modo da mantenere il sistema osservabile, sostituibile e coerente con lo stack i3Guild. La Figura 2.2 riporta lo schema di alto livello dell'architettura.

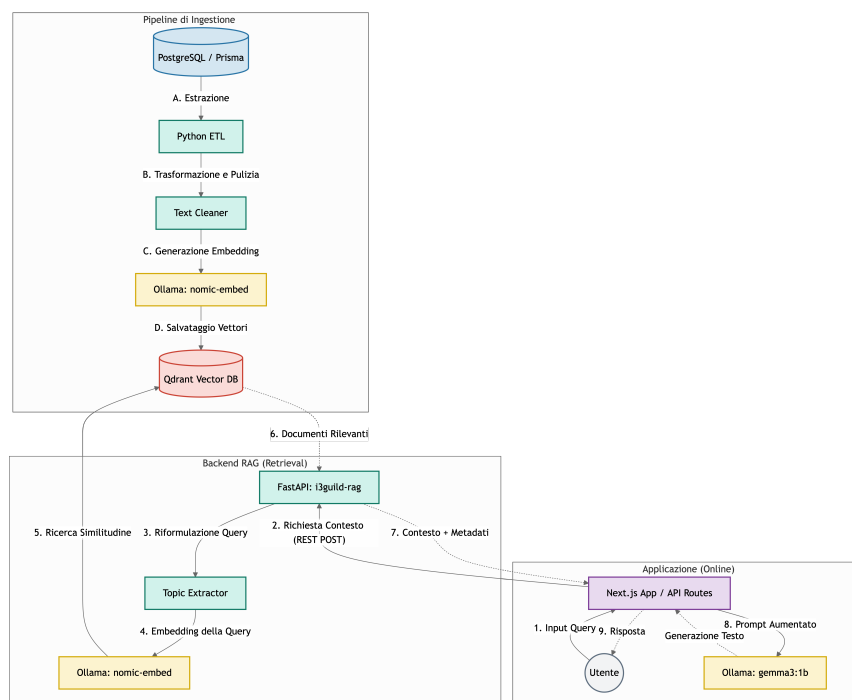


Figura 2.2: Schema architetturale del sistema RAG on-premise per i3Guild. Il flusso di ingestione (ETL) popola Qdrant a partire da PostgreSQL; il flusso di retrieval serve le query di Next.js tramite FastAPI.

2.5.1 Componenti principali

I componenti non sono indipendenti solo dal punto di vista del deployment: ciascuno risponde a un requisito emerso nelle sezioni precedenti. PostgreSQL rimane la sorgente autoritativa, Qdrant rende interrogabile il corpus per similarità, Ollama evita dipendenze cloud e FastAPI espone un confine applicativo semplice verso Next.js.

Ingestione e pre-processing (rag/ingest.py) Pipeline batch Python che esegue le fasi ETL descritte nella Sezione 2.5.3. Acquisisce i documenti da PostgreSQL, normalizza il testo, esegue il chunking, genera gli embedding tramite Ollama e indicizza su Qdrant.

Strategia di chunking Sulla base dell'analisi dei dati (Sezione 2.2.3), viene adottato l'approccio a *documento consolidato* (1 Gilda $\leftrightarrow$ 1 documento vettoriale): tutti i campi testuali rilevanti sono concatenati in un unico documento con separatori strutturati. Per i rari outlier che eccedono 8.000 token viene applicata una *soft truncation*. Questo approccio preserva il contesto semantico integrale e semplifica la gestione di upsert e cancellazioni su Qdrant.

Database vettoriale (Qdrant) Collection principale `i3guild_knowledge` con vettori a 768 dimensioni, metrica coseno e indice HNSW. Ogni punto porta un payload JSON con metadati indicizzati per il filtering (Sezione 2.5.2).

Orchestrazione RAG (Haystack 2.x) Il container `i3guild_rag` espone pipeline Haystack per ingestione e retrieval. I componenti tipizzati (`DocumentCleaner`, `DocumentSplitter`, `OllamaDocumentEmbedder`, `QdrantDocumentStore`) sono collegati in pipeline dichiarative.

Inference LLM (Ollama) Server di inferenza locale; modello di chat adottato: `gemma3:1b`, selezionato sulla base del benchmark sperimentale condotto su dodici modelli (Sezione 2.4.4).

Servizio API (FastAPI) Microservizio Python che espone l'endpoint `POST /rag/retrieve` (porta 8000, mappata a 8002 su host). Restituisce documenti con score di similarità e campi diagnostici.

Osservabilità e sicurezza Logging centralizzato, metriche di latenza e utilizzo risorse; autenticazione tramite Keycloak (già presente in i3Guild); cifratura a riposo per i dataset sensibili.

2.5.2 Progettazione della collection Qdrant

La collection `i3guild_knowledge` è il punto in cui la conoscenza estratta da i3Guild viene trasformata in indice semantico. La configurazione, riportata nella Tabella 2.7, riflette il modello di embedding selezionato e privilegia una metrica adatta alla similarità testuale.

Tabella 2.7: Parametri di configurazione della collection Qdrant.

Parametro	Valore	Motivazione
<code>vectors.size</code>	768	Output di <code>nomic-embed-text</code>
<code>vectors.distance</code>	<code>Cosine</code>	Standard per similarità semantica testuale
<code>hnsw_config.m</code>	16 (default)	Compromesso velocità/precisione
<code>hnsw_config.ef_construct</code>	100 (default)	Qualità dell'indice in fase di build

Ogni punto nella collection porta un payload JSON con i campi elencati nella Tabella 2.8. I campi marcati come indicizzati abilitano il pre-filtering, caratteristica distintiva di Qdrant che applica i filtri sul payload *prima* della ricerca vettoriale, garantendo che il calcolo del top- K sia effettuato solo sul sottoinsieme filtrato.

Gli ID dei punti sono **UUID deterministici** generati tramite hash di `guild_id + field_source + chunk_index`, garantendo l'idempotenza degli upsert: lo stesso documento produce sempre lo stesso ID, prevenendo la creazione di duplicati in caso di re-indicizzazione.

Tabella 2.8: Struttura del payload Qdrant per la collection `i3guild_knowledge`.

Campo	Tipo	Descrizione	Indicizzato
<code>guild_id</code>	integer	ID della gilda sorgente	Sì
<code>guild_name</code>	keyword	Nome della gilda	Sì
<code>epoch_id</code>	integer	ID dell'epoca di appartenenza	Sì
<code>epoch_start</code>	datetime	Data inizio epoca	Sì
<code>epoch_end</code>	datetime	Data fine epoca (nullable)	Sì
<code>participation_mode</code>	keyword	FullRemote / Hybrid / OnSite	Sì
<code>is_individual</code>	bool	Gilda individuale	Sì
<code>owner_id</code>	keyword	ID del proprietario	Sì
<code>participants</code>	keyword[]	Nomi dei partecipanti	Sì
<code>participant_count</code>	integer	Numero di partecipanti	No
<code>source_text</code>	text	Testo originale del chunk	No
<code>last_synced</code>	datetime	Timestamp ultima sincronizzazione	No

2.5.3 Pipeline di ingestione ETL

La pipeline ETL ha il compito di mantenere allineata la rappresentazione vettoriale con i dati applicativi. Lo script `rag/ingest.py` esegue le seguenti fasi:

1. **Extract:** connessione a PostgreSQL e query SQL con JOIN su `Epoch`, `GuildUserRelation` ed `ExternalUser` per estrarre tutti i campi semantici delle Gilde insieme ai nomi dei partecipanti.
2. **Transform:** applicazione di `DocumentCleaner` (stripping HTML dai campi WYSIWYG come `journal`), normalizzazione degli spazi, rimozione di segnaposti residui.
3. **Embed:** generazione degli embedding tramite `OllamaDocumentEmbedder` (modello `nomic-embed-text`, endpoint interno Docker `http://ollama:11434`).
4. **Load:** scrittura su Qdrant tramite `DocumentWriter` con policy `DuplicatePolicy.OVERWRITE`.

La struttura della pipeline Haystack è riportata in Appendice B, Listato B.1. Nel capitolo principale è sufficiente evidenziare la sequenza logica Extract-Transform-Embed-Load, perché il dettaglio del codice non modifica la scelta progettuale.

Con lo stack Docker attivo, l'ingestione può essere avviata da host tramite il comando `docker-compose exec i3guild_rag python rag/ingest.py`.

2.5.4 Pipeline di retrieval e logica di ranking

La pipeline di retrieval (`rag/api.py`) usa l'indice Qdrant per recuperare i documenti più pertinenti alla query e restituisce al livello applicativo un contesto già filtrato. Il sistema non adotta un valore `top_k` fisso: una soglia rigida rischierebbe infatti di includere documenti marginali nelle query semplici o di tagliare informazioni utili nelle query più ampie. Per questo viene applicato un approccio adattivo in tre stadi:

1. **Soglia di similarità** (default: cosine score ≥ 0.7): esclude documenti semanticamente non pertinenti.

2. **Rilevamento del gap**: se il salto di score tra due documenti consecutivi supera una soglia configurabile (default: 0.08), la lista viene troncata al confine naturale della rilevanza.
3. **Cap di sicurezza** (default: ≤ 10 documenti): limite massimo per controllare la dimensione del contesto passato al modello generativo.

Query rewriting e Topic-First Search

Nel contesto i3Guild, molte query contengono parole ricorrenti come “gilda”, “fondare” o “consigli”. Questi termini descrivono l’interazione con il sistema, ma non il tema informativo cercato; se mantenuti nella query, possono dominare lo spazio degli embedding e produrre risultati generici. Il *query rewriting*, discusso anche in framework come RQ-RAG [5], viene quindi utilizzato per isolare il topic rilevante prima del retrieval. L’approccio implementato tramite la funzione `extract_topic_query()` opera in tre passaggi:

1. Normalizzazione e rimozione della punteggiatura.
2. Filtraggio di un insieme di circa 80 *domain stopwords* (DOMAIN_STOPWORDS): termini specifici del dominio i3Guild e stopwords italiane generiche.
3. Estrazione del topic residuo; se diverso dalla query originale, la ricerca viene eseguita sul topic pulito (*topic-first strategy*).

La Tabella 2.9 illustra l’effetto del rewriting su query campione.

Tabella 2.9: Effetto del query rewriting su esempi di query utente.

Query originale	Topic estratto	Risultato atteso
“vorrei fondare una gilda sulla musica, cosa mi consigli?”	musica	Gilda a tema musicale
“intelligenza artificiale”	intelligenza artificiale	(nessun rewriting)
“chi ha partecipato al progetto di design?”	design	Gilda con partecipanti nel settore design

2.5.5 Endpoint API di riferimento

Il risultato della pipeline viene esposto tramite un endpoint FastAPI, che costituisce il contratto tra il servizio RAG e l’applicazione Next.js. L’endpoint principale è il seguente:

POST /rag/retrieve

Un esempio di request body e la struttura della response sono riportati in Appendice B, Listati B.2 e B.3.

I campi `total_candidates`, `threshold_used`, `cutoff_reason` e `topic_query` (presente solo quando il rewriting è attivo) sono utili per il debug e la valutazione sperimentale del sistema (Capitolo 5).

2.6 Integrazione con i3Guild

L'integrazione del sistema RAG nell'infrastruttura i3Guild esistente è pensata per ridurre l'impatto sul sistema già operativo. PostgreSQL, Next.js, Keycloak e Kong mantengono il proprio ruolo; il sistema RAG si aggiunge come insieme di servizi nel file `dev_containers/docker-compose.yml`. Questa scelta permette di introdurre il retrieval semantico senza modificare il modello dati principale né il flusso di autenticazione.

2.6.1 Servizi Docker aggiuntivi

La Tabella 2.10 elenca i container aggiuntivi introdotti dal sistema RAG.

Tabella 2.10: Servizi Docker introdotti dall'integrazione RAG in i3Guild.

Container	Immagine	Porta host	Ruolo
qdrant	qdrant:latest	6333, 6334	Database vettoriale; Web UI su <code>/dashboard</code>
ollama	ollama:latest	11434	Server di inferenza LLM ed embedding
i3guild_rag	Build locale	8002	FastAPI + Haystack (retrieval e ingestione)
postgres	già presente	già presente	Source of truth relazionale (nessuna modifica)

2.6.2 Variabili d'ambiente

Tabella 2.11: Variabili d'ambiente introdotte dalla componente RAG.

Variabile	Valore di default	Descrizione
QDRANT_URL	<code>http://localhost:6333</code>	Endpoint REST Qdrant
QDRANT_COLLECTION_NAME	<code>i3guild_knowledge</code>	Collection principale
OLLAMA_BASE_URL	<code>http://localhost:11434</code>	Endpoint Ollama
OLLAMA_EMBEDDING_MODEL	<code>nomic-embed-text</code>	Modello di embedding
OLLAMA_CHAT_MODEL	<code>gemma3:1b</code>	Modello LLM di chat (selezionato da benchmark)
EMBEDDING_DIMENSIONS	<code>768</code>	Dimensione vettore
RAG_API_URL	<code>http://i3guild_rag:8000</code>	Endpoint interno del servizio RAG

2.6.3 Integrazione lato Next.js

Lato applicazione Next.js, l'integrazione è concentrata nel servizio TypeScript `src/lib/services/rag.service.ts`, che espone la funzione `searchRAG(query, filters?, options?)`. La funzione invoca l'endpoint Python `/rag/retrieve` con i parametri di retrieval dinamico e restituisce un array vuoto in caso di

irraggiungibilità del servizio Python. In questo modo il chatbot può degradare in modo controllato, senza interrompere il resto dell'esperienza applicativa.

Le chiamate a Ollama per la generazione del testo utilizzano un agent **undici** personalizzato con `headersTimeout: 300s` per evitare il `HeadersTimeoutError` che si manifesta al primo avvio, quando Ollama carica il modello in memoria (latenza tipica di 60–120 s su CPU-only).

2.6.4 Sincronizzazione dei dati

PostgreSQL rimane la *source of truth*; Qdrant è una vista derivata e deve essere mantenuto sincronizzato con i dati applicativi. Sono state considerate tre strategie, in ordine crescente di complessità: re-indicizzazione completa, sincronizzazione incrementale e aggiornamento event-driven. Per il prototipo la strategia raccomandata è il **full re-index periodico**: lo script `rag/ingest.py` riscrive l'intera collection su base giornaliera o settimanale. La scelta è semplice da implementare, riduce il rischio di inconsistenze e rimane sostenibile perché l'embedding è eseguito localmente, senza costi a consumo per token. L'evoluzione verso un *sync incrementale* basato sul campo `@updatedAt` di Prisma, o verso un meccanismo *event-driven* attivato dalle operazioni CRUD, è lasciata come sviluppo futuro.

Capitolo 3

Implementazione del Sistema RAG

Il capitolo descrive l'implementazione del sistema RAG integrato in i3Guild. L'obiettivo non è ripetere le scelte architetturali già motivate nel Capitolo 2, ma mostrare come tali scelte siano state tradotte in componenti eseguibili: una pipeline di ingestione per popolare Qdrant, un microservizio FastAPI per il retrieval e la generazione, un'integrazione applicativa lato Next.js e un workflow agentic per accompagnare l'utente nella proposta di nuove Gilde. Quest'ultimo viene documentato come soluzione sperimentale: è stato implementato ed esplorato, ma non è stato mantenuto come scelta finale a causa del costo computazionale più elevato e del vantaggio limitato rispetto a un flusso RAG più classico a parità di risorse hardware.

La realizzazione è distribuita su tre repository applicativi. Il primo è `i3guild`, che contiene l'applicazione web Next.js e il database PostgreSQL [53]. Il secondo è `i3guild-embedding-pipeline`, responsabile dell'estrazione dei dati, della normalizzazione testuale, della generazione degli embedding e dell'upsert su Qdrant. Il terzo è `i3guild-rag`, un microservizio Python che espone le API di retrieval e chat. Questa separazione riduce l'accoppiamento tra applicazione e componente RAG: i3Guild rimane la sorgente autoritativa dei dati, mentre Qdrant e Ollama vengono trattati come servizi specializzati [55, 49].

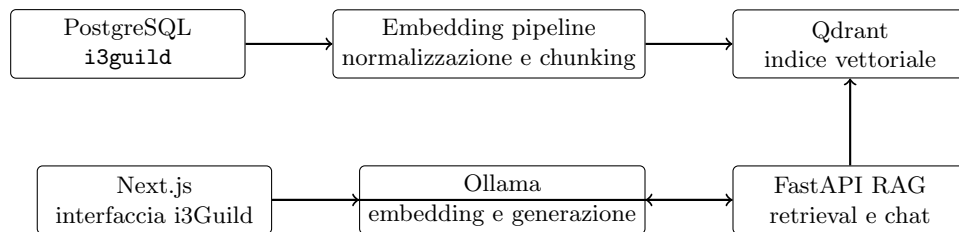


Figura 3.1: Flusso implementativo end-to-end tra sorgente dati, indicizzazione, retrieval e generazione.

3.1 Struttura dei repository e responsabilità

La suddivisione dei moduli segue il flusso dei dati. PostgreSQL contiene le Gilde, le epoche e le relazioni con gli utenti; la pipeline di embedding legge queste informazioni e produce documenti Haystack [40]; Qdrant memorizza i vettori e i metadati; il microservizio RAG interroga l'indice e, quando richiesto, invoca Ollama per generare la risposta. L'applicazione Next.js consuma il servizio tramite un client HTTP dedicato [60].

Tabella 3.1: Componenti implementativi principali.

Repository / modulo	File principali	Responsabilità
i3guild	services/rag.service.ts	Client TypeScript verso il microservizio RAG; gestione dello streaming e degradazione controllata in caso di errore.
-embedding-pipeline	pipeline/main.py, pipeline/ingest_sources.py	Estrazione da PostgreSQL, normalizzazione, chunking, embedding con Ollama e scrittura su Qdrant.
-rag	app/main.py, app/rag.py, app/pipeline.py, app/ollama_service.py	API FastAPI, pipeline Haystack di query, retrieval dinamico, chiamate a Ollama e streaming NDJSON.
-rag/app/agent	orchestrator.py, state.py, models.py	Workflow agentic deterministico esplorato per ricerca, ideazione e compilazione progressiva della proposta di Gilda; non adottato nella soluzione finale.

La scelta di mantenere l'ingestione fuori dal servizio di query evita che operazioni batch e richieste interattive competano nello stesso processo. La pipeline di embedding può essere eseguita manualmente, schedulata o avviata in container dedicato; il servizio RAG, invece, rimane ottimizzato per rispondere a richieste HTTP a bassa latenza. La containerizzazione isola dipendenze e servizi di supporto [41].

3.2 Pipeline di ingestione

La pipeline di ingestione è implementata in `i3guild-embedding-pipeline/pipeline/main.py`. Il processo viene avviato come job: verifica le dipendenze, legge le sorgenti dati, produce documenti Haystack, genera gli embedding tramite Ollama e scrive i risultati in Qdrant [40, 49, 55]. In caso di errore in una dipendenza essenziale, ad esempio PostgreSQL o Qdrant non raggiungibile, il processo termina con codice diverso da zero. Questo comportamento è utile in ambiente Docker o CI, perché rende immediatamente visibile il fallimento dell'indicizzazione [41].

3.2.1 Configurazione e controlli preliminari

La configurazione avviene tramite variabili d'ambiente. Le più rilevanti sono `DB_URL`, `QDRANT_URL`, `QDRANT_COLLECTION_NAME`, `OLLAMA_BASE_URL`, `OLLAMA_EMBEDDING_MODEL` ed `EMBEDDING_DIMENSIONS`.

Il modulo legge anche parametri operativi come `RECREATE_INDEX`, `INGEST_MAX_DOCUMENT_CHARS` e `INGEST_CHUNK_OVERLAP_CHARS`.

Tabella 3.2: Parametri principali della pipeline di ingestione.

Variabile	Ambito	Uso nell'implementazione
<code>DB_URL</code>	PostgreSQL	Connessione alla sorgente autoritativa delle Gilde.
<code>QDRANT_URL</code>	Qdrant	Endpoint del vector database usato in scrittura.
<code>QDRANT_COLLECTION_NAME</code>	Qdrant	Nome della collection da creare o aggiornare.
<code>OLLAMA_BASE_URL</code>	Ollama	Endpoint locale per generare gli embedding.
<code>OLLAMA_EMBEDDING_MODEL</code>	Ollama	Modello usato da <code>OllamaDocumentEmbedder</code> .
<code>EMBEDDING_DIMENSIONS</code>	Indice vettoriale	Dimensione dei vettori salvati in Qdrant.
<code>RECREATE_INDEX</code>	Operatività	Ricrea la collection quando serve una reindicizzazione completa.
<code>INGEST_MAX_DOCUMENT_CHARS</code>	Chunking	Limite oltre il quale un documento viene suddiviso.

Prima dell'ingestione vengono eseguiti tre controlli. Il primo apre una connessione a PostgreSQL ed esegue una query minimale. Il secondo verifica la raggiungibilità di Qdrant e crea la collection se non esiste. Il terzo chiama l'endpoint `/api/tags` di Ollama per verificare che il server sia raggiungibile e che il modello di embedding sia configurato. La collection Qdrant viene creata con distanza coseno e dimensione pari a `EMBEDDING_DIMENSIONS`; nel caso del modello `nomic-embed-text` il valore utilizzato è 768 [26].

3.2.2 Estrazione dei dati da PostgreSQL

La funzione `fetch_all_guilds()` estrae le informazioni semantiche delle Gilde tramite una query SQL con join su `Epoch`, `GuildUserRelation` ed `ExternalUser`. Il risultato include i campi testuali principali della Gilda, i riferimenti all'epoca, la modalità di partecipazione, l'indicazione di Gilda individuale e la lista dei partecipanti. Il listato completo dei campi estratti è riportato in Appendice B, Listato B.4.

La query reale include anche una sottoquery per aggregare i partecipanti, combinando utenti interni ed esterni. Questo passaggio è importante perché i nomi dei partecipanti sono utili sia come metadati filtrabili sia come contesto testuale per le domande dell'utente.

3.2.3 Normalizzazione e costruzione dei documenti

Ogni riga estratta viene trasformata in uno o più oggetti `haystack.Document`. Attraverso la funzione `create_documents_from_row()` compone un documento testuale con sezioni marcate, ad esempio `[SOMMARIO]`, `[OBIETTIVI]`, `[RISCHI]`, `[RISULTATI RAGGIUNTI]` e `[PARTECIPANTI]`. I campi HTML

vengono normalizzati con `BeautifulSoup` [58]: il codice rimuove i tag, conserva il testo e compatta gli spazi. Per il retrieval serve un testo pulito e stabile, non una resa editoriale. Uno schema semplificato della costruzione del documento è riportato in Appendice B, Listato B.5.

La strategia descritta nel Capitolo 2 prevedeva un documento consolidato per Gilda. L'implementazione mantiene questa impostazione come caso normale, ma introduce una soglia di sicurezza: `INGEST_MAX_DOCUMENT_CHARS`. Se il testo supera il limite, la funzione `split_text_with_overlap()` lo divide in chunk con sovrapposizione controllata. In questo modo gli outlier non impediscono l'ingestione e, allo stesso tempo, i chunk successivi mantengono il riferimento alla Gilda tramite l'ingestione. La granularità dei chunk condiziona qualità del retrieval e quantità di contesto disponibile per il modello generativo [6].

Gli identificativi dei documenti sono deterministici: `guild-{guild_id}-chunk-{index}`. La scelta rende idempotente la reindicizzazione: rieseguire la pipeline sulla stessa Gilda produce gli stessi ID e permette a Qdrant di sovrascrivere i punti esistenti invece di duplicarli. I metadati associati includono `guild_id`, `guild_name`, `epoch_id`, `is_individual`, `participation_mode`, `participants`, `chunk_index` e `chunk_count`.

3.2.4 Sorgenti aggiuntive

Oltre a PostgreSQL, la pipeline supporta due sorgenti opzionali: file JSON e scraping delle Gilde pubblicate. Il modulo `ingest_sources.py` espone `load_guilds_from_json()` e `stream_guilds_from_scraper()`. Nel primo caso il file viene letto come lista di dizionari; nel secondo caso il codice usa il web scraper incluso nel repository per iterare sulle Gilde pubbliche e, se richiesto, arricchirle con dati estratti dagli articoli collegati.

Questa estensione non cambia il contratto con Qdrant: anche i dati esterni vengono trasformati in documenti Haystack con ID deterministici, contenuto testuale e metadati. Per le sorgenti non autoritative la policy di duplicazione predefinita è più conservativa: PostgreSQL usa `OVERWRITE`, mentre JSON e scraper usano `SKIP` salvo configurazione esplicita. La ragione è pratica: il database applicativo è la fonte principale, mentre le sorgenti esterne possono essere incomplete o già presenti nell'indice.

3.2.5 Embedding e scrittura in Qdrant

La fase finale costruisce una pipeline Haystack minimale composta da `OllamaDocumentEmbedder` e `DocumentWriter`. L'embedder usa il modello configurato in `OLLAMA_EMBEDDING_MODEL`, con `num_ctx=8192`; il writer persiste i documenti nel `QdrantDocumentStore`. La configurazione essenziale della pipeline è riportata in Appendice B, Listato B.6.

La pipeline non usa un componente di cleaning Haystack separato, perché la normalizzazione dei campi avviene prima della costruzione dei documenti. Questa scelta rende più esplicito il formato testuale indicizzato e consente di controllare quali campi entrano nel corpus.

3.3 Microservizio RAG

Il microservizio RAG è implementato con FastAPI nel repository `i3guild-rag`. Espone endpoint per retrieval, chat standard, chat in streaming e workflow agentico. La configurazione è centralizzata in `app/settings.py` tramite `pydantic-settings`; i valori di default permettono lo sviluppo locale, mentre gli ambienti Docker possono sovrascriverli con variabili d'ambiente [42, 54].

Tabella 3.3: Endpoint principali esposti dal microservizio RAG.

Endpoint	Formato	Responsabilità
POST <code>/rag/retrieve</code>	JSON	Recupera documenti semanticamente pertinenti e restituisce metadati di diagnostica sul taglio dei risultati.
POST <code>/chat/reply</code>	JSON	Esegue retrieval, costruisce il prompt aumentato e restituisce una risposta completa.
POST <code>/chat/stream</code>	NDJSON	Espone la chat incrementale, separando eventi RAG, token, warning ed errori.
POST <code>/chat/agentlic</code>	JSON	Applica l'orchestrazione deterministica e aggiorna lo stato della proposta.
POST <code>/chat/agentlic/stream</code>	NDJSON	Restituisce decisione agentica, risultati RAG e risposta in forma incrementale.

3.3.1 Avvio, middleware e gestione errori

Il file `app/main.py` definisce l'applicazione FastAPI e registra CORS, rate limiting e handler globale delle eccezioni. Il rate limiter usa `slowapi` con limite predefinito di 100 richieste al minuto per IP sugli endpoint standard e 60 richieste al minuto sugli endpoint agentici. Il sistema non tratta tutti gli errori allo stesso modo: se il messaggio suggerisce una dipendenza non raggiungibile, ad esempio Qdrant o Ollama, la risposta HTTP è 503; negli altri casi viene restituito 500.

Per ridurre il rischio operativo in ambienti con risorse limitate, il servizio controlla la memoria disponibile prima delle chiamate a modelli locali. La funzione `get_model_memory_requirement()` stima il requisito RAM in base al nome del modello; `get_ram_warning()` produce un warning se la memoria libera è inferiore alla soglia. Il warning non blocca la richiesta, ma viene propagato nella risposta quando disponibile.

3.3.2 Endpoint di retrieval

L'endpoint POST `/rag/retrieve` riceve una query, parametri di retrieval e filtri opzionali. Il modello Pydantic `RetrieveRequest` definisce i valori di default: `top_k=20`, `score_threshold=0.7`, `max_documents=10` e `gap_threshold=0.08`. La risposta include i documenti selezionati, il numero di candidati iniziali, la soglia usata, il motivo del taglio, l'eventuale query riscritta e gli eventuali filtri estratti automaticamente. Lo schema della richiesta è riportato in Appendice B, Listato B.7.

L'endpoint è volutamente sottile: valida l'input, esegue il controllo RAM e delega la logica a `retrieve_documents()`. Questa separazione rende più semplice testare il retrieval senza dover passare da FastAPI.

3.3.3 Pipeline Haystack di query

La pipeline di query è costruita in `app/pipeline.py`. Il `QdrantDocumentStore` punta alla collection configurata e usa embedding di dimensione 768 con similarità coseno. Il componente `OllamaTextEmbedder` trasforma la query utente in vettore; il `QdrantEmbeddingRetriever` esegue la ricerca in Qdrant [40, 55]. Il listato della pipeline è riportato in Appendice B, Listato B.8.

La pipeline è inizializzata in modo lazy e protetta da un lock. Alla prima richiesta viene creato l'oggetto Haystack e vengono predisposti gli indici sui payload Qdrant per i campi usati più spesso nei filtri: `epoch_id`, `is_individual`, `participation_mode`, `participants` e `guild_name`. Le chiamate successive riusano la stessa pipeline in memoria. La ricerca vettoriale su Qdrant usa indici approssimati per nearest neighbor; HNSW è il riferimento adottato per questa classe di problemi [22].

3.4 Logica di retrieval

La funzione `retrieve_documents()` in `app/rag.py` contiene la parte più specifica del sistema. Il retrieval non è una semplice chiamata a Qdrant: prima della ricerca la query viene normalizzata e, quando possibile, riscritta; dopo la ricerca i risultati vengono deduplicati e filtrati con una logica dinamica. Questa struttura segue il principio RAG di separare conoscenza parametrica e recupero esplicito di documenti esterni [20].

3.4.1 Riscrittura della query e filtri

La funzione `extract_topic_query()` rimuove parole frequenti nel dominio i3Guild, come "gilda", "fondare" o "consigli", quando non contribuiscono al tema ricercato. Se dalla frase "vorrei fondare una gilda sulla musica" viene estratto il topic "musica", la ricerca viene eseguita su tale topic. La risposta conserva la query riscritta nel campo `topic_query`, così il frontend può mostrarla o usarla per diagnostica.

In parallelo, `extract_filters_from_query()` cerca vincoli espressi in linguaggio naturale: epoca, Gilde individuali o di gruppo, modalità di partecipazione e partecipanti. Se l'utente passa anche filtri espliciti, il servizio li combina con quelli estratti tramite un operatore **AND**. Questo permette di usare la ricerca vettoriale senza perdere vincoli strutturati presenti nei metadati.

3.4.2 Cache e deduplicazione

Per evitare interrogazioni ripetute identiche, `retrieve_documents()` mantiene una cache in memoria con TTL configurabile tramite `rag_cache_ttl_seconds`. La chiave include query effettiva, parametri di retrieval e filtri. Si tratta di una cache locale al processo, sufficiente per lo scenario prototipale; in un deployment multi replica andrebbe sostituita con una cache condivisa o disabilitata per evitare differenze tra istanze.

Dopo il recupero da Qdrant, il servizio rimuove duplicati su tre livelli. Prima controlla l'ID del documento, poi usa una fingerprint testuale normalizzata e infine confronta i testi con `difflib.SequenceMatcher`. La soglia fuzzy è configurabile con `rag_dedup_fuzzy_threshold`. Questa parte è stata aggiunta perché il corpus può contenere documenti equivalenti provenienti da sorgenti diverse o chunk molto simili generati dalla sovrapposizione.

3.4.3 Selezione dinamica dei documenti

La selezione finale avviene in `filter_documents_dynamically()`. I documenti vengono ordinati per score, filtrati con una soglia minima e poi tagliati quando il calo relativo tra due score consecutivi supera `gap_threshold`. Se il numero di risultati resta troppo alto, viene applicato `max_documents`.

La risposta indica il motivo del taglio: `threshold`, `relative_gap`, `max_cap` o `exhausted`. Uno schema della logica di taglio è riportato in Appendice B, Listato B.9.

Questa scelta evita un limite `top_k` fisso. Per query molto specifiche il sistema restituisce pochi documenti; per query più ampie può mantenere più contesto, entro il cap configurato. Il risultato è più stabile per l'utente e riduce la probabilità di passare al modello generativo documenti debolmente pertinenti. Ridurre contesto non pertinente è utile anche per mitigare risposte non fondate sulle fonti recuperate [30].

3.5 Generazione con Ollama

Il modulo `app/ollama_service.py` gestisce le chiamate al modello di chat. Le funzioni principali sono `generate_chat_reply()` per risposte complete e `generate_chat_reply_stream()` per risposte in streaming. Entrambe usano l'endpoint `/api/chat` di Ollama [49].

3.5.1 Risoluzione del modello

Il modello configurato in `OLLAMA_LLM_MODEL` viene confrontato con la lista dei modelli installati ottenuta da `/api/tags`. Il codice accetta sia corrispondenze esatte sia varianti con tag, ad esempio `:latest`. Se la configurazione è ambigua o il modello non è installato, il servizio restituisce un errore esplicito. Questa verifica evita fallimenti meno leggibili durante la generazione.

La lista dei modelli disponibili viene mantenuta in cache per pochi secondi (`ollama_model_cache_ttl_seconds`), così il servizio non interroga Ollama a ogni richiesta. La cache è locale al processo e ha solo funzione di riduzione dell'overhead.

3.5.2 Costruzione del prompt aumentato

Prima di inviare la richiesta a Ollama, `_prepare_messages()` costruisce la lista dei messaggi. Il system prompt, se presente, viene aggiunto per primo. La cronologia conversazionale viene limitata agli ultimi `max_history_messages`, in modo da controllare la dimensione del contesto. Se il retrieval ha restituito documenti, il testo viene aggiunto al messaggio utente dentro una sezione esplicita: `--- CONTESTO RAG ---`. L'aumento del prompt con documenti recuperati è il meccanismo con cui RAG rende disponibili al modello informazioni esterne senza modificare i pesi [20]. Il formato del messaggio aumentato è riportato in Appendice B, Listato B.10.

Ogni documento viene troncato a `rag_max_chars_per_doc` caratteri. Il troncamento è intenzionale: una singola Gilda molto lunga non deve consumare tutta la finestra di contesto né impedire al modello di vedere altre fonti.

3.5.3 Streaming NDJSON

Per la chat interattiva il servizio espone `POST /chat/stream`. Prima invia un evento `rag` con i metadati del retrieval, poi inoltra gli eventi generati da Ollama. La risposta HTTP usa il media type `application/x-ndjson`; ogni riga è un oggetto JSON indipendente. Questo formato è semplice da consumare lato frontend e permette di distinguere token, warning, errori e fine stream. Un esempio di sequenza NDJSON è riportato in Appendice B, Listato B.11.

3.6 Workflow agentic

Il sistema ha esplorato anche una modalità agentic, esposta dagli endpoint `/chat/agentic` e `/chat/agentic/stream`. La documentazione progettuale iniziale prevedeva un agente a due step basato su decisione LLM e sintesi. L'implementazione corrente adotta invece un orchestratore deterministico in `app/agent/orchestrator.py`. La scelta è più adatta a un prototipo con modelli locali piccoli: riduce la variabilità, rende testabile il comportamento e limita il numero di chiamate al modello.

Questa parte dell'implementazione deve quindi essere interpretata come una soluzione esplorata e testata, non come il nucleo della soluzione finale. Le prove condotte hanno mostrato che il modello agentic richiede più risorse computazionali, soprattutto in termini di latenza e disponibilità di memoria, mentre l'approccio RAG standard risulta più robusto e conveniente a parità di hardware. Per questo motivo la componente agentic è stata mantenuta come evidenza sperimentale e come possibile direzione futura, ma non come scelta architetturale vincente per l'implementazione finale.

3.6.1 Stato conversazionale

Lo stato dell'agente è rappresentato da `AgentState`. Contiene la fase corrente, il topic della proposta, il campo del form in compilazione e una bozza strutturata. Le fasi previste sono `triage`, `benchmark`, `ideation`, `form`, `finalize`, `search` e `answer_direct`. La bozza contiene i campi minimi necessari per formalizzare una proposta: nome, descrizione, rischi, risorse e risultati attesi. Il modello Pydantic dello stato è riportato in Appendice B, Listato B.12.

Tabella 3.4: Fasi del workflow agentic e comportamento associato.

Fase	Attivazione	Comportamento
<code>triage</code>	Primo messaggio o stato non definito	Classifica l'intento dell'utente e sceglie il ramo successivo.
<code>benchmark</code>	Richiesta di proposta su un tema	Cerca Gilde precedenti utili come confronto iniziale.
<code>ideation</code>	Benchmark completato	Propone una direzione progettuale e prepara la compilazione.
<code>form</code>	Raccolta dei dati della proposta	Valida progressivamente nome, descrizione, rischi, risorse e output attesi.
<code>finalize</code>	Campi minimi compilati	Restituisce una bozza coerente con il form i3Guild.
<code>search</code>	Domanda su Gilde esistenti	Usa il retrieval senza avviare il flusso di proposta.
<code>answer_direct</code>	Messaggi semplici o saluti	Risponde senza interrogare Qdrant.

Ogni risposta agentic include anche un `AgentTrace`, cioè un blocco diagnostico con decisione, query riscritta, ragionamento sintetico, numero di documenti recuperati e filtri estratti. Questo dato non serve al modello, ma al frontend e alla valutazione: permette di capire perché il sistema ha scelto una ricerca, una risposta diretta o la prosecuzione del questionario.

3.6.2 Regole di orchestrazione

L'orchestratore usa espressioni regolari per classificare il messaggio utente. Messaggi di saluto vengono gestiti con risposta diretta; richieste su Gilde esistenti attivano la ricerca; richieste di proposta o creazione avviano una fase di benchmark, cioè una ricerca di esperienze precedenti sul topic individuato. Quando l'utente entra nella fase di compilazione, il sistema valida il campo corrente e passa al successivo solo se il contenuto è sufficiente.

Questa logica è meno flessibile di un agente completamente governato da LLM, ma ha due vantaggi concreti. Primo, il comportamento è riproducibile. Secondo, il sistema può guidare l'utente in un flusso formale senza affidare a un modello piccolo la gestione implicita dello stato. Tuttavia, anche in questa forma più controllata, il flusso agentico introduce complessità e overhead superiori rispetto al percorso RAG standard. Il compromesso è quindi utile come sperimentazione, ma non sufficiente a giustificare l'adozione nel prototipo finale.

3.6.3 Eventi di streaming agentico

La versione streaming dell'endpoint agentico restituisce eventi NDJSON. Il primo evento comunica la decisione dell'agente; il secondo espone i dati RAG; seguono inizio stream, token e fine stream. Anche se l'orchestratore corrente produce la risposta completa prima di emettere i token, il contratto HTTP resta compatibile con una futura implementazione in cui la sintesi venga generata token per token da Ollama. Un esempio di stream agentico è riportato in Appendice B, Listato B.13.

3.7 Integrazione lato Next.js

L'applicazione `i3Guild` accede al microservizio tramite `src/lib/services/rag.service.ts`. Il client centralizza l'URL del servizio, legge `I3GUILD_RAG_URL` o `RAG_API_URL`, serializza le richieste e normalizza gli errori. Questa scelta evita di distribuire chiamate HTTP dirette al servizio RAG in più componenti React o route API [60].

3.7.1 Client di retrieval

La funzione `searchRAGWithMeta()` invoca `/rag/retrieve` e restituisce sia documenti sia metadati. In caso di errore non solleva un'eccezione: restituisce una lista vuota e un `cutoff_reason` pari a `error`. Per la sola ricerca semantica questo comportamento è corretto: il chatbot non deve bloccarsi se il servizio Python è temporaneamente non raggiungibile. La chiamata TypeScript essenziale è riportata in Appendice B, Listato B.14.

La funzione `searchRAG()` è un wrapper più semplice che restituisce solo i documenti. Viene mantenuta per i punti dell'applicazione che non hanno bisogno della diagnostica.

3.7.2 Client chat e modalità agentica sperimentale

Per la generazione sono disponibili `generateRAGChatReply()`, `generateRAGChatStream()` e `generateAgenticRAGChatStream()`. La prima usa l'endpoint non streaming `/chat/reply`; le altre restituiscono direttamente la `Response`, lasciando al chiamante la lettura incrementale dello stream NDJSON.

La variante agentica sperimentale passa anche `agent_state`. Questo consente al frontend di mantenere lo stato tra turni di conversazione senza imporre al microservizio una sessione server-side. Il servizio RAG rimane quindi stateless: ogni richiesta contiene la cronologia e lo stato

necessari per calcolare il turno successivo. La scelta è utile per isolare la sperimentazione, ma non modifica il flusso RAG standard adottato come riferimento finale.

3.8 Generazione della bozza di Gilda

Accanto alla chat RAG, i3Guild include un flusso per generare una bozza di Gilda a partire dalla conversazione. La funzionalità è documentata come *Guild Draft Generation* e si attiva dal pulsante "Proponi bozza" nella chat. Il flusso è separato dal retrieval: usa la conversazione come input, costruisce un prompt istruttivo, invoca Ollama e prova a estrarre un oggetto JSON compatibile con il form di creazione.

Il formato atteso è **GuildDraft**. Include nome, sommario, obiettivi, risultati attesi, rischi, opportunità, budget, risorse, journal, modalità di partecipazione, numero minimo e massimo di partecipanti, indicazione di Gilda individuale e avatar. Una volta generata, la bozza viene codificata in Base64 e passata alla pagina `/guild/create` tramite query string. La pagina di creazione decodifica il parametro, lo combina con lo stato applicativo e mostra il form precompilato.

Il punto rilevante dell'implementazione è il fallback. Se non esiste una conversazione, se il modello restituisce JSON non valido o se mancano campi obbligatori, il codice genera una bozza template valida. Il flusso utente quindi non dipende completamente dalla qualità della generazione: l'LLM migliora la bozza quando riesce, ma non è l'unico meccanismo per arrivare al form.

3.9 Aspetti operativi

L'implementazione include alcune scelte operative che non cambiano la logica RAG ma ne migliorano l'affidabilità durante l'uso locale.

3.9.1 Timeout e dipendenze lente

Ollama può impiegare diversi secondi, o più di un minuto su CPU, per caricare un modello non ancora residente in memoria. Per questo il servizio Python usa timeout più lunghi nelle chiamate streaming, mentre la documentazione lato Next.js prevede un **undici Agent** con **headersTimeout** e **bodyTimeout** estesi. Senza questa configurazione, il primo messaggio dopo l'avvio del container rischia di fallire anche se il modello è installato correttamente.

3.9.2 System prompt e modello derivato

La documentazione del progetto descrive anche una strategia opzionale per stabilizzare il system prompt in Ollama. Invece di inviare il messaggio **system** a ogni chiamata, il sistema può creare un modello derivato con prompt incorporato tramite `/api/create`. Il nome del modello contiene un hash del prompt; se il prompt cambia, cambia anche il nome del modello derivato.

Questa tecnica non è un vero riuso della KV Cache del modello. Il vantaggio misurato riguarda soprattutto la dimensione del payload e la stabilità configurativa: nei benchmark registrati, il payload JSON si riduce in modo marcato, mentre il numero di token valutati da Ollama rimane invariato. Per la tesi questo punto è utile perché distingue un'ottimizzazione architetturale effettiva da un'ottimizzazione solo apparente.

3.9.3 Tracciabilità

Il servizio espone dati diagnostici in più punti: `cutoff_reason`, `topic_query`, `extracted_filters`, `warning RAM` e `AgentTrace`. Questi campi non sono solo utili per il debug. Permettono di ricostruire il comportamento del sistema durante la valutazione sperimentale: quale query è stata effettivamente cercata, quanti documenti sono stati recuperati, perché alcuni risultati sono stati scartati e quale fase agentica ha prodotto la risposta.

3.10 Sintesi del capitolo

Il sistema implementato combina componenti semplici, ma separati con precisione. La pipeline di embedding prepara l'indice a partire da PostgreSQL e da sorgenti opzionali; il microservizio RAG espone retrieval e generazione; Qdrant gestisce la ricerca vettoriale; Ollama mantiene l'inferenza in locale; Next.js integra le funzioni nel flusso utente. Le parti più specifiche del lavoro sono la normalizzazione del corpus i3Guild, il retrieval dinamico con query rewriting e deduplicazione. Il workflow agentico deterministico è stato invece trattato come una linea sperimentale: ha permesso di valutare un flusso più guidato per la proposta di nuove Gilde, ma non è risultato preferibile all'approccio classico nelle condizioni hardware disponibili. Queste scelte rendono il prototipo coerente con i vincoli di privacy e di risorse discussi nei capitoli precedenti, lasciando aperti margini di evoluzione su sincronizzazione incrementale, agenti più flessibili e valutazione sistematica della qualità delle risposte.

Capitolo 4

Ottimizzazione e Adattamento al Dominio

Il capitolo analizza le strategie adottate per adattare il sistema RAG al dominio i3Guild senza trasformare il prototipo in un progetto di addestramento del modello. Questa scelta deriva da un vincolo concreto: non è disponibile un glossario tecnico aziendale validato e non esiste, allo stato attuale, un dataset supervisionato sufficiente per un fine-tuning robusto. Il corpus utilizzabile è costituito principalmente dai dati delle Gilde, dai relativi metadati, dalle descrizioni testuali e dai contenuti pubblicati nel tempo.

L'adattamento al dominio viene quindi affrontato in modo non parametrico. Il sistema non modifica i pesi del modello, ma interviene sulla qualità del retrieval, sulla normalizzazione delle query, sull'uso dei metadati, sulla costruzione del prompt e sulla configurazione dei modelli locali. In questo modo il sistema rimane aggiornabile, eseguibile on-premise e coerente con i vincoli architetturali definiti nei capitoli precedenti.

La prima parte del capitolo descrive i vincoli sperimentali: assenza di un glossario formale, disponibilità parziale dei dati aziendali, risorse hardware limitate e necessità di mantenere separati il sistema applicativo e la componente RAG. La seconda parte approfondisce le ottimizzazioni implementate: query rewriting, stopword di dominio, filtri strutturati, deduplicazione, chunking, soglie dinamiche di retrieval e gestione del contesto generativo. La terza parte riguarda l'estensione agantica, trattata come evoluzione del flusso RAG tradizionale e non come sostituzione dell'architettura di base.

Il capitolo include inoltre una sezione di sperimentazione preliminare su strategie alternative di adattamento. Tecniche come LoRA, QLoRA, fine-tuning e interventi a tempo di inferenza vengono considerate in termini di fattibilità, requisiti e limiti, senza renderle parte del sistema aziendale integrato. Questa analisi serve a delimitare il perimetro del prototipo e a motivare perché, nel lavoro svolto, l'adattamento non parametrico rappresenti la scelta più solida.

La chiusura del capitolo definisce la configurazione finale da portare alla valutazione del Capitolo 5: parametri di retrieval, criteri di selezione dei documenti, impostazioni del modello locale e modalità agantica. In questo modo il Capitolo 5 può concentrarsi sulla valutazione del sistema risultante, evitando di riaprire decisioni architetturali già motivate.

Capitolo 5

Analisi Sperimentale e Valutazione dei Risultati

Il capitolo valuta il sistema ottenuto dopo le scelte progettuali, implementative e di ottimizzazione descritte nei capitoli precedenti. La valutazione non ha l'obiettivo di dimostrare la superiorità assoluta di un modello, ma di verificare se il prototipo risponde in modo adeguato al caso d'uso aziendale: supportare la consultazione delle Gilde e la proposta di nuove iniziative tramite un sistema RAG eseguibile on-premise.

La prima parte del capitolo definisce il dataset di validazione. In assenza di una ground truth aziendale preesistente, il set di test può essere costruito a partire dai dati disponibili sulle Gilde: domande fattuali, domande tematiche, richieste di confronto tra iniziative e richieste di supporto alla proposta di una nuova Gilda. Ogni domanda deve essere associata a documenti attesi o a criteri espliciti di correttezza, così da rendere la valutazione ripetibile.

La seconda parte riguarda le metriche. Per il retrieval possono essere usati indicatori come hit rate, precision@k e MRR, limitatamente ai casi in cui sia possibile definire documenti rilevanti attesi. Per la generazione è più opportuno combinare metriche automatiche e valutazione manuale: fedeltà alle fonti recuperate, pertinenza rispetto alla domanda, completezza della risposta e presenza di allucinazioni. Nel caso della modalità agentica, la valutazione deve considerare anche la capacità del sistema di guidare l'utente nella costruzione di una proposta coerente.

La terza parte confronta configurazioni diverse del sistema. Il confronto principale riguarda pipeline RAG tradizionale e flusso agentico introdotto nel prototipo. Eventuali confronti tra modelli locali, soglie di retrieval o configurazioni di prompt vengono trattati come analisi di supporto, non come nuovi obiettivi sperimentali indipendenti. Questa distinzione mantiene il capitolo allineato al perimetro definito nel Capitolo 4.

La parte finale discute i casi di fallimento: documenti non recuperati, query ambigue, risposte corrette ma incomplete, risposte non sufficientemente ancorate alle fonti e limiti dei modelli locali piccoli. Tale analisi è necessaria per distinguere i problemi dovuti al corpus, quelli dovuti al retrieval e quelli dovuti alla generazione.

Conclusioni e Sviluppi Futuri

Le conclusioni sintetizzano il percorso svolto: dallo studio delle tecniche RAG e dei modelli locali, alla progettazione del sistema per i3Guild, fino all'implementazione del prototipo e alla valutazione delle sue configurazioni principali. Il contributo centrale della tesi non consiste nell'addestramento di un nuovo modello, ma nell'integrazione di un sistema RAG enterprise on-premise in un contesto aziendale reale, con vincoli di privacy, risorse e manutenibilità.

La sezione conclusiva dovrà evidenziare tre risultati. Primo, la possibilità di usare il corpus delle Gilde come base di conoscenza interrogabile in linguaggio naturale. Secondo, il ruolo delle ottimizzazioni non parametriche, come query rewriting, filtri strutturati, deduplicazione e soglie dinamiche, nel rendere il retrieval più adatto al dominio. Terzo, il valore e i limiti dell'estensione agentica, utile per guidare l'utente nella proposta di nuove Gilde ma ancora dipendente dalla qualità del corpus e dal comportamento dei modelli locali.

I limiti principali riguardano l'assenza di un glossario aziendale validato, la mancanza di una ground truth estesa, la dimensione ridotta dei modelli eseguibili localmente e la natura preliminare delle sperimentazioni su tecniche di adattamento più invasive. Questi limiti non invalidano il prototipo, ma definiscono con precisione il perimetro entro cui interpretarne i risultati.

Gli sviluppi futuri possono essere organizzati su tre direzioni: arricchimento incrementale della base di conoscenza, costruzione di un dataset di valutazione più ampio e validato con stakeholder aziendali, e sperimentazione più approfondita di strategie come fine-tuning leggero, LoRA/QLoRA o orchestrazione agentica più flessibile. Tali estensioni potranno essere considerate solo dopo aver consolidato il sistema RAG di base e il processo di valutazione.

Appendice A

Annotazioni e Convenzioni

Questa appendice raccoglie le convenzioni usate nella descrizione del sistema RAG. L'obiettivo è evitare ripetizioni nei capitoli principali e rendere esplicito il significato di nomi, formati e parametri ricorrenti.

A.1 Repository e componenti

Tabella A.1: Convenzioni di denominazione dei repository.

Nome	Significato
i3guild	Applicazione web principale, basata su Next.js, PostgreSQL e Keycloak.
i3guild-embedding-pipeline	Job Python che estrae, normalizza e indicizza il corpus delle Gilde.
i3guild-rag	Microservizio FastAPI che espone retrieval, chat RAG e workflow agentico.

A.2 Formato dei documenti indicizzati

I documenti salvati in Qdrant derivano dai record applicativi di i3Guild. Ogni documento contiene un testo normalizzato e un insieme di metadati usati per filtri, diagnostica e deduplicazione.

A.3 Eventi NDJSON

Gli endpoint streaming restituiscono una sequenza di righe JSON indipendenti. La scelta del formato NDJSON permette al frontend di elaborare subito i metadati RAG e poi aggiornare l'interfaccia man mano che arrivano i token.

Tabella A.2: Metadati principali associati ai documenti indicizzati.

Campo	Uso
<code>guild_id</code>	Identificativo della Gilda sorgente.
<code>guild_name</code>	Nome della Gilda, usato anche nei risultati mostrati all'utente.
<code>epoch_id</code>	Identificativo dell'epoca a cui appartiene la Gilda.
<code>participants</code>	Lista dei partecipanti, utile come metadato filtrabile.
<code>participation_mode</code>	Modalità di partecipazione, ad esempio in presenza, remota o ibrida.
<code>is_individual</code>	Indica se la proposta riguarda una Gilda individuale.
<code>chunk_index</code>	Posizione del chunk quando una Gilda viene suddivisa.
<code>chunk_count</code>	Numero totale di chunk generati per la stessa Gilda.

Tabella A.3: Tipi di evento usati dagli endpoint streaming.

Evento	Significato
<code>rag</code>	Contiene documenti recuperati, filtri, query riscritta e motivo del taglio dei risultati.
<code>agent_decision</code>	Indica la decisione presa dall'orchestratore agentico.
<code>stream_start</code>	Segnala l'inizio della risposta generata.
<code>token</code>	Contiene un frammento incrementale della risposta.
<code>stream_end</code>	Chiude lo stream e, nella modalità agentica, può includere lo stato aggiornato.
<code>error</code>	Comunica un errore gestibile senza interrompere il contratto dello stream.

Appendice B

Codice Utilizzato

Questa appendice raccoglie i listati richiamati nei capitoli principali. Nel testo sono mantenute le scelte progettuali, le decisioni implementative e il razionale; i frammenti di codice sono spostati qui per non interrompere la lettura.

B.1 Listati richiamati nel Capitolo 2

Listing B.1: Struttura della pipeline di ingestione Haystack 2.x (pseudocodice).

```
from haystack import Pipeline
from haystack.components.preprocessors import DocumentCleaner ,
    DocumentSplitter
from haystack.components.writers import DocumentWriter
from haystack.integrations.components.embedders.ollama import
    OllamaDocumentEmbedder
from haystack.integrations.document_stores.qdrant import
    QdrantDocumentStore

qdrant_store = QdrantDocumentStore(url="http://qdrant:6333" ,
                                   collection_name="i3guild_knowledge" ,
                                   embedding_dim=768)

indexing_pipeline = Pipeline()
indexing_pipeline.add_component("cleaner" , DocumentCleaner())
indexing_pipeline.add_component("splitter" , DocumentSplitter(
    split_by="word" , split_length=500 , split_overlap=50))
indexing_pipeline.add_component("embedder" , OllamaDocumentEmbedder(
    model="nomic-embed-text" ,
    url="http://ollama:11434/api/embed" ,
    generation_kwargs={"num_ctx": 8192}))
indexing_pipeline.add_component("writer" , DocumentWriter(
    document_store=qdrant_store ,
```

```

        policy=DuplicatePolicy.OVERWRITE))

indexing_pipeline.connect("cleaner.documents", "splitter.documents")
indexing_pipeline.connect("splitter.documents", "embedder.documents")
indexing_pipeline.connect("embedder.documents", "writer.documents")

indexing_pipeline.run({"cleaner": {"documents": raw_docs}})

```

Listing B.2: Esempio di request body per l'endpoint `/rag/retrieve`.

```

{
  "query":          "vorrei fondare una gilda sulla musica",
  "top_k":          20,
  "score_threshold": 0.5,
  "max_documents":  10,
  "gap_threshold":  0.08,
  "filters":        null
}

```

Listing B.3: Struttura della response dell'endpoint `/rag/retrieve`.

```

{
  "documents":      [ { "content": "...", "score": 0.85,
                        "meta": { "guild_name": "...",
                                  "participants": [...] } } ],
  "total_candidates": 15,
  "threshold_used":  0.5,
  "cutoff_reason":   "gap",
  "topic_query":     "musica"
}

```

B.2 Listati richiamati nel Capitolo 3: pipeline di ingestione

Listing B.4: Campi principali estratti da PostgreSQL per l'ingestione.

```

SELECT
  g.id AS guild_id ,
  g.name AS guild_name ,
  g.summary ,
  g.goals ,
  g.opportunities ,
  g.outcome ,
  g.risks ,
  g."otherResources" AS other_resources ,
  g.budget ,

```

```

    g."achievedResultsDescription" AS achieved_results_description ,
    g.journal ,
    g."isIndividualGuild" AS is_individual ,
    g."participationMode" AS participation_mode ,
    e.id AS epoch_id ,
    e."startDate" AS epoch_start ,
    e."endDate" AS epoch_end
FROM "Guild" g
LEFT JOIN "Epoch" e ON g."epochId" = e.id

```

Listing B.5: Schema semplificato della costruzione del documento testuale.

```

parts = [ f" [GUILD: {guild_name}]" ]

if row.get("summary"):
    parts.append( f" [SOMMARIO] {clean_html(row.get('summary'))}" )
if row.get("goals"):
    parts.append( f" [OBIETTIVI] {clean_html(row.get('goals'))}" )
if row.get("risks"):
    parts.append( f" [RISCHI] {clean_html(row.get('risks'))}" )
if participants:
    parts.append( f" [PARTECIPANTI] {', '.join(participants)}" )

full_text = "\n".join(parts)

```

Listing B.6: Pipeline Haystack usata per embedding e scrittura.

```

document_store = QdrantDocumentStore(
    url=QDRANT_URL,
    index=QDRANT_COLLECTION_NAME,
    embedding_dim=EMBEDDING_DIMENSIONS,
    similarity="cosine",
    recreate_index=RECREATE_INDEX,
)

embedder = OllamaDocumentEmbedder(
    model=OLLAMA_EMBEDDING_MODEL,
    url=OLLAMA_BASE_URL,
    timeout=300,
    generation_kwargs={"num_ctx": 8192},
)

writer = DocumentWriter(document_store=document_store, policy=policy)
indexing_pipeline = Pipeline()
indexing_pipeline.add_component("embedder", embedder)
indexing_pipeline.add_component("writer", writer)
indexing_pipeline.connect("embedder.documents", "writer.documents")

```

B.3 Listati richiamati nel Capitolo 3: retrieval e generazione

Listing B.7: Schema Pydantic della richiesta di retrieval.

```
class RetrieveRequest(BaseModel):
    query: str
    top_k: int = 20
    score_threshold: float = 0.7
    max_documents: int = 10
    gap_threshold: float = 0.08
    filters: dict | None = None
```

Listing B.8: Pipeline Haystack per la ricerca semantica.

```
document_store = QdrantDocumentStore(
    url=QDRANT_URL,
    index=QDRANT_COLLECTION_NAME,
    embedding_dim=768,
    similarity="cosine",
    recreate_index=False,
)
text_embedder = OllamaTextEmbedder(
    model=OLLAMA_EMBEDDING_MODEL,
    url=OLLAMA_BASE_URL,
)
retriever = QdrantEmbeddingRetriever(document_store=document_store)

pipeline = Pipeline()
pipeline.add_component("text_embedder", text_embedder)
pipeline.add_component("retriever", retriever)
pipeline.connect("text_embedder.embedding", "retriever.query_embedding")
)
```

Listing B.9: Logica semplificata di taglio dinamico dei risultati.

```
scored_docs.sort(key=lambda x: x[0], reverse=True)
above_threshold = [
    d for score_value, d in scored_docs
    if score_value >= score_threshold
]

filtered = [above_threshold[0]]
for i in range(1, len(above_threshold)):
    prev = float(getattr(above_threshold[i - 1], "score", 0.0) or 0.0)
    cur = float(getattr(above_threshold[i], "score", 0.0) or 0.0)
    relative_drop = (prev - cur) / prev if prev > 0 else 1.0
    if relative_drop > gap_threshold:
```

```

        cutoff_reason = "relative_gap"
        break
    filtered.append(above_threshold[i])

```

Listing B.10: Formato del messaggio utente aumentato con contesto RAG.

```

final_message = (
    f"{message}\n\n"
    f"-----CONTESTO-RAG-----\n"
    f"Usa queste informazioni per rispondere.\n"
    f"{final_context}\n"
    f"-----FINE-CONTESTO-----"
)

```

Listing B.11: Esempio di eventi NDJSON prodotti dalla chat standard.

```

{"type": "rag", "data": {"documents": [...], "cutoff_reason": "relative_gap"}}
{"type": "stream_start", "model": "qwen2.5:1.5b"}
{"type": "token", "content": "La"}
{"type": "token", "content": " gilda"}
{"type": "stream_end"}

```

B.4 Listati richiamati nel Capitolo 3: workflow agentico e frontend

Listing B.12: Modello di stato usato dal workflow agentico.

```

class ProposalDraft(BaseModel):
    name: str | None = None
    description: str | None = None
    risks: str | None = None
    resources: str | None = None
    outcomes: str | None = None

class AgentState(BaseModel):
    phase: AgentPhase = "triage"
    topic: str | None = None
    current_field: ProposalField | None = None
    proposal: ProposalDraft = Field(default_factory=ProposalDraft)

```

Listing B.13: Esempio di eventi NDJSON prodotti dalla modalità agentica.

```

{"type": "agent_decision", "decision": "benchmark", "rewritten_query": "musica"}

```

```
{ "type": "rag", "retrieved": 5, "docs": [] }  
{ "type": "stream_start", "model": "deterministic-orchestrator" }  
{ "type": "token", "content": "Ho " }  
{ "type": "stream_end", "agent_state": {...} }
```

Listing B.14: Chiamata TypeScript al retrieval RAG.

```
const response = await fetch(`${RAG_API_URL}/rag/retrieve`, {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    query,  
    filters,  
    top_k: 20,  
    score_threshold: options?.scoreThreshold ?? 0.7,  
    max_documents: options?.maxDocuments ?? 10,  
  }),  
});
```

Bibliografia

Riferimenti Bibliografici

- [1] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint arXiv:2310.11511* (2023). URL: <https://arxiv.org/abs/2310.11511>.
- [2] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI Feedback”. In: *arXiv preprint arXiv:2212.08073* (2022). URL: <https://arxiv.org/abs/2212.08073>.
- [3] Rishi Bommasani et al. “On the Opportunities and Risks of Foundation Models”. In: *arXiv preprint arXiv:2108.07258* (2021). URL: <https://arxiv.org/abs/2108.07258>.
- [4] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901. URL: <https://arxiv.org/abs/2005.14165>.
- [5] Chi-Min Chan, Xinyu Chen, Rui Yin et al. “RQ-RAG: Learning to Rewrite Queries for Retrieval Augmented Generation”. In: *arXiv preprint* (2024).
- [6] Jiawei Chen et al. “Benchmarking Large Language Models in Retrieval-Augmented Generation”. In: *arXiv preprint arXiv:2309.01431* (2023). URL: <https://arxiv.org/abs/2309.01431>.
- [7] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: <https://arxiv.org/abs/2305.14314>.
- [8] Darren Edge et al. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization”. In: *arXiv preprint arXiv:2404.16130* (2024). URL: <https://arxiv.org/abs/2404.16130>.
- [9] European Parliament e Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council — General Data Protection Regulation*. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [10] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)*. Rapp. tecn. L 119. Official Journal of the European Union, 2016, pp. 1–88. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679>.

- [11] Elias Frantar et al. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. In: *International Conference on Learning Representations*. 2023. URL: <https://arxiv.org/abs/2210.17323>.
- [12] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997* (2024). URL: <https://arxiv.org/abs/2312.10997>.
- [13] Sepp Hochreiter e Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [14] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: (2022). URL: <https://arxiv.org/abs/2106.09685>.
- [15] Intré S.r.l. *10 pratiche per costruire una Learning Organization: il modello Intré*. <https://www.intre.it/2024/01/15/10-pratiche-per-costruire-una-learning-organization-il-modello-intre/>. Ultimo accesso: maggio 2026. 2024.
- [16] Intré S.r.l. *Archivio Gilde*. <https://www.intre.it/gilde/>. Ultimo accesso: aprile 2026. 2024.
- [17] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38. DOI: 10.1145/3571730.
- [18] Albert Q. Jiang et al. “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825* (2023). URL: <https://arxiv.org/abs/2310.06825>.
- [19] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [20] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [21] Shih-Yang Liu et al. “DoRA: Weight-Decomposed Low-Rank Adaptation”. In: *International Conference on Machine Learning (ICML)*. 2024. URL: <https://arxiv.org/abs/2402.09353>.
- [22] Yury A. Malkov e Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.
- [23] Niklas Muennighoff, Nils Reimers, Lasse Magne et al. *MTEB: Massive Text Embedding Benchmark*. arXiv preprint arXiv:2210.07316. 2023. URL: <https://arxiv.org/abs/2210.07316>.
- [24] Niklas Muennighoff et al. “MTEB: Massive Text Embedding Benchmark”. In: *arXiv preprint arXiv:2210.07316* (2022). URL: <https://arxiv.org/abs/2210.07316>.
- [25] Tsendsuren Munkhdalai, Manaal Faruqui e Siddharth Gopal. “Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention”. In: *arXiv preprint arXiv:2404.07143* (2024). URL: <https://arxiv.org/abs/2404.07143>.
- [26] Zach Nussbaum et al. “Nomic Embed: Training a Reproducible Long Context Text Embedder”. In: *arXiv preprint arXiv:2402.01613* (2024). URL: <https://arxiv.org/abs/2402.01613>.

- [27] OpenAI. *GPT-4 Technical Report*. Rapp. tecn. OpenAI, 2023. URL: <https://arxiv.org/abs/2303.08774>.
- [28] Qwen Team e Alibaba Cloud. *Qwen2 Technical Report*. 2024. URL: <https://arxiv.org/abs/2407.10671>.
- [29] Peter M. Senge. *The Fifth Discipline: The Art and Practice of the Learning Organization*. Edizione italiana: *La quinta disciplina*, Sperling & Kupfer, 1990. Doubleday/Currency, 1990.
- [30] Kurt Shuster et al. “Retrieval Augmentation Reduces Hallucination in Conversation”. In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 3784–3803. URL: <https://arxiv.org/abs/2104.07567>.
- [31] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023). URL: <https://arxiv.org/abs/2302.13971>.
- [32] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [33] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837. URL: <https://arxiv.org/abs/2201.11903>.
- [34] Shi-Qi Yan et al. “Corrective Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2401.15884* (2024). URL: <https://arxiv.org/abs/2401.15884>.
- [35] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.
- [36] Shenglai Zeng, Jiankun Zhang, Pengfei He et al. “The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)”. In: *arXiv preprint arXiv:2402.16893* (2024). URL: <https://arxiv.org/abs/2402.16893>.

Sitografia

- [37] AIMultiple Research. *RAG Frameworks: LangChain vs LangGraph vs LlamaIndex*. Benchmark comparativo di 5 framework RAG su 100 query standardizzate. 2026. URL: <https://research.aimultiple.com/rag-frameworks/>.
- [38] Anthropic. *Claude 4 Model Card*. 2025. URL: <https://www.anthropic.com/claude/claude-4>.
- [39] Athenic AI. *LangChain vs LlamaIndex vs Haystack: RAG Framework Comparison*. 2025. URL: <https://getathenic.com/blog/langchain-vs-llamaindex-vs-haystack-rag-frameworks>.
- [40] deepset GmbH. *Haystack: Open Source NLP Framework to Build Production-Ready LLM Applications*. 2023. URL: <https://haystack.deepset.ai>.
- [41] Docker Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/>.
- [42] FastAPI Contributors. *FastAPI Documentation*. 2024. URL: <https://fastapi.tiangolo.com/>.

- [43] Google DeepMind. *Gemini 2.5: Our Most Intelligent Model*. 2025. URL: <https://deepmind.google/technologies/gemini/>.
- [44] Google Research. *TurboQuant: Redefining AI efficiency with extreme compression*. Consultato il: 27-03-2026. Presentato a ICLR 2026. Mar. 2026. URL: <https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/>.
- [45] Nirant Kasliwal. *pgvector vs Qdrant: Results from the 1M OpenAI Benchmark*. 2024. URL: <https://nirantk.com/writing/pgvector-vs-qdrant/>.
- [46] LangChain AI. *LangGraph: Build Resilient Language Agents as Graphs*. 2024. URL: <https://langchain-ai.github.io/langgraph/>.
- [47] Meta AI. *Llama 4: The Next Generation of Open Foundation Models*. 2025. URL: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [48] Mistral AI. *Mistral Large 3*. 2025. URL: <https://mistral.ai/news/mistral-large-3/>.
- [49] Ollama Contributors. *Ollama: Get Up and Running With Large Language Models Locally*. 2023. URL: <https://ollama.com>.
- [50] OpenAI. *Learning to Reason with LLMs*. 2024. URL: <https://openai.com/index/learning-to-reason-with-llms/>.
- [51] Pelican. *Haystack vs LangChain: Explained*. 2025. URL: <https://pelican.io/blog/haystack-vs-langchain/>.
- [52] Ethan Perez. *RAG Evaluation Series: Validating the RAG Performance of LangChain vs Haystack*. Tonic.ai Blog. 2024. URL: <https://www.tonic.ai/blog/rag-evaluation-series-validating-the-rag-performance-of-langchain-vs-haystack>.
- [53] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/>.
- [54] Pydantic Contributors. *Pydantic Documentation*. 2024. URL: <https://docs.pydantic.dev/>.
- [55] Qdrant Contributors. *Qdrant: Vector Database for the Next Generation of AI Applications*. 2023. URL: <https://qdrant.tech>.
- [56] Qdrant Team. *Vector Search Benchmarks*. Benchmark ufficiale Qdrant su dataset ann-benchmarks; aggiornato gennaio/giugno 2024. 2024. URL: <https://qdrant.tech/benchmarks/>.
- [57] Qwen Team e Alibaba Cloud. *Qwen3 Technical Report*. 2025. URL: <https://qwenlm.github.io/blog/qwen3/>.
- [58] Leonard Richardson. *Beautiful Soup Documentation*. 2024. URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
- [59] Tiger Data. *Pgvector vs. Qdrant: Benchmarking Open-Source Vector Databases for AI at Scale*. Benchmark su 50M embedding Cohere a 768 dimensioni con fork ANN-Benchmarks. 2025. URL: <https://www.tigerdata.com/blog/pgvector-vs-qdrant>.
- [60] Vercel. *Next.js Documentation*. 2024. URL: <https://nextjs.org/docs>.